



Bangladesh University of Engineering & Technology

Course No: EEE 212

Course Title: Numerical Technique Laboratory

Project Report

Project title:

2D-Heat Conduction Equation Solve in MATLAB

Submitted To:

Dr. Maruf Ahmed, Assistant Professor, Department of EEE, BUET

Md. Kamrul Hasan, Lecturer, Department of EEE, BUET

Submitted By:

Arnob Pal

Student ID: 2306095

Ziaur Rahman Zihan

Student ID: 2306096

Department : EEE

Section : B1

Level : 2

Term : 1

Date of Submission: 07/04/2026

Introduction:

The study of heat transfer plays a central role in engineering and scientific analysis, as temperature distribution directly influences the performance, safety, and efficiency of countless systems. From electronic circuit design and power systems to aerospace structures and energy devices, understanding how heat propagates within a material is essential. In this context, the two-dimensional (2D) heat equation serves as a fundamental mathematical model that describes how temperature varies spatially within a domain under given physical conditions. The 2D heat equation provides a powerful framework for analyzing steady-state and transient thermal behavior. However, for most real-world applications involving irregular geometries, non-uniform materials, and complex boundary conditions, obtaining analytical solutions becomes extremely difficult or even impossible. This limitation necessitates the use of advanced numerical techniques that can approximate the solution with high accuracy.

Among various numerical methods, the Finite Element Method (FEM) stands out as one of the most versatile and robust approaches for solving partial differential equations such as the heat equation. FEM operates by discretizing the continuous domain into a finite number of smaller subregions, known as elements, which are interconnected through nodes. This discretization transforms the original continuous problem into a system of algebraic equations that can be solved computationally.

In the context of 2D heat transfer, the domain is typically divided into triangular or quadrilateral elements forming a mesh. Within each element, the temperature distribution is approximated using carefully selected shape functions (basis functions). These functions allow the temperature field to be expressed in terms of nodal values, thereby reducing the problem to determining a finite set of unknowns. By applying physical laws and variational principles, a global system of linear equations is constructed, representing the thermal behavior of the entire domain.

The accuracy of FEM lies in its ability to capture local variations in temperature while maintaining global consistency across the domain. Additionally, FEM offers significant flexibility in handling complex geometries, varying material properties,

and different types of boundary conditions, making it highly suitable for practical engineering problems.

In this project, we utilize the computational capabilities of MATLAB, specifically through App Designer, to develop a complete FEM-based solver for the 2D steady-state heat equation. The objective is not only to implement the numerical algorithm but also to gain a deep understanding of the underlying mathematical formulation, including discretization, matrix assembly, and solution procedures. Through this work, we aim to analyze temperature distributions in a systematic and efficient manner, providing insights that are directly applicable to real-world thermal systems.

Ultimately, this project bridges the gap between theoretical heat transfer concepts and practical computational implementation, demonstrating how modern numerical methods can be employed to solve complex engineering problems with precision and efficiency.

Objectives:

- 1.** Develop a robust FEM-based numerical framework in MATLAB for solving the two-dimensional steady-state heat equation, incorporating a systematic discretization of complex geometries into triangular elements and deriving the necessary element-level formulations.
- 2.** Formulate and implement the weak form of the heat equation, including the derivation of shape functions, the B matrix, and the element stiffness matrices, ensuring accurate representation of temperature variations within each element.
- 3.** Assemble the global system of equations from individual element contributions, while accurately applying various boundary conditions such as Dirichlet and convection types, to reflect realistic physical scenarios.
- 4.** Validate the numerical model by comparing computed results against analytical solutions, benchmark examples, or experimental data, evaluating accuracy, convergence, and reliability of the FEM implementation.

5. Investigate the impact of computational parameters—including mesh density, element size, and discretization strategy—on the precision of temperature predictions and the overall efficiency of the solver.
6. Apply the developed FEM solver to real-world engineering problems, such as thermal management in electronic devices, heat distribution in building structures, or thermal behavior in composite materials, interpreting results to provide practical insights.
7. Analyze and interpret simulation outcomes to identify patterns, temperature gradients, hotspots, and effects of varying material and boundary properties, linking numerical results to physical phenomena.
8. To gain practical hands-on experience in numerical programming using MATLAB, including mesh generation, matrix assembly, linear algebra, and engineering visualization.

Theory:

Governing Equation: The Heat Conduction Equation

The physical problem we are solving is described by a Partial Differential Equation (PDE). For a 2D steady-state (time-independent) problem with no internal heat generation, the governing equation is the Laplace equation for temperature $T(x, y)$:

$$\frac{\partial}{\partial x} \left(k_x \cdot \frac{\partial T}{\partial x} \right) + \frac{\partial}{\partial y} \left(k_y \cdot \frac{\partial T}{\partial y} \right) = 0$$

Where k_x and k_y are the thermal conductivities in the x and y directions respectively.

In this project, the material is assumed isotropic, so $k_x = k_y = k$.

This equation simply states that the net heat flowing into any small control volume must equal zero in steady state — whatever heat comes in must go out.

Boundary Conditions

The governing heat equation is subject to a combination of Dirichlet & Robin boundary conditions, simulating a system with a constant energy source and convective environmental losses.

Dirichlet Boundary Condition (Prescribed Temperature)

On the left boundary (Γ_D), the temperature is fixed to represent a continuous heat source or a perfectly controlled thermal interface. This is expressed as:

$$T(0, y) = T_{\text{boundary}}$$

This condition assumes the thermal reservoir has infinite capacity, maintaining the face at a constant temperature regardless of the internal flux.

Robin Boundary Condition (Convective Heat Flux)

All remaining external boundaries (Γ_R), including the right edge ($x = 70$), the horizontal top/bottom surfaces, and the geometric step at $x = 10$, are governed by Newton's Law of Cooling. This Robin boundary condition accounts for the convective heat exchange between the solid surface and the ambient air:

$$-k\nabla T \cdot n = h(T - T_{\infty})$$

Where:

- n : The outward-pointing unit normal vector at the boundary.
- h : The convective heat transfer coefficient ($W/m^2 \cdot K$).
- T_{∞} : The ambient fluid temperature, fixed at 20°C.
- $-k\nabla T \cdot n$: The conductive heat flux (q) exiting the domain.

By applying these conditions simultaneously, the model transitions from a simple gradient problem to a Third-Kind Boundary Value Problem. This configuration is highly representative of industrial heat sinks or structural components where one

end is mounted to a heat-generating element while the rest of the geometry is cooled by passive or forced air circulation.

Finite Element Method (FEM) — Overview

The Finite Element Method works by dividing the complex domain into many small simple shapes called elements. In this project, the elements are triangles with 3 nodes, called T3 elements. The key idea is:

1. Divide the domain into small triangles (meshing).
2. On each triangle, approximate the unknown temperature $T(x,y)$ using simple polynomial shape functions.
3. Use the governing equation (in weak/variational form) to derive element-level equations relating temperatures at the element's nodes.
4. Assemble all element equations into one big global system of equations.
5. Apply boundary conditions and solve the system for the unknown temperatures.

T3 Triangular Element and Linear Shape Functions

A T3 element is the simplest 2D triangular element used in FEM. It has:

- 3 nodes, each at the triangle's corners.
- Coordinates: $(x_1, y_1), (x_2, y_2), (x_3, y_3)$
- Unknown temperatures at nodes: T_1, T_2, T_3
- Goal: Approximate the temperature $T(x, y)$ at any point inside the triangle using these nodal values.

Temperature Interpolation

Inside the element, temperature is interpolated as a linear combination of nodal values and shape functions:

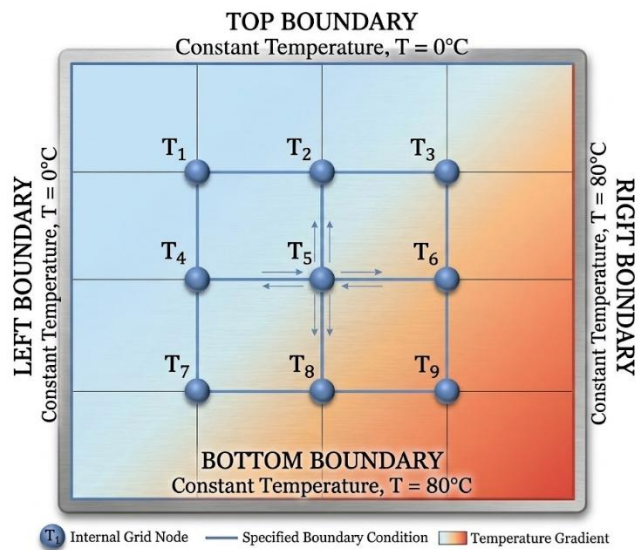
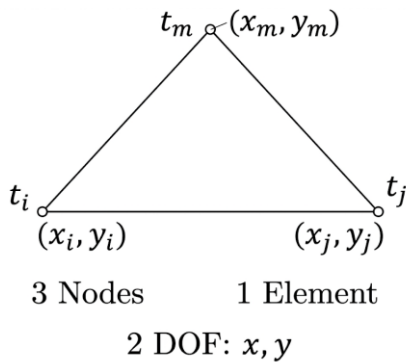
$$T(x, y) = N_1(x, y)T_1 + N_2(x, y)T_2 + N_3(x, y)T_3$$

Where:

- N_1, N_2, N_3 are the **shape functions**.
- Each N_i satisfies:

$$N_i(\text{node } j) = \delta_{ij}, \text{ where } i, j = 1, 2, 3$$

This ensures that at a node, the shape function corresponding to that node equals 1, and the others are 0.



Linear Shape Functions (T3)

For a T3 element, the shape functions are linear in x and y :

$$N_i(x, y) = \frac{a_i + b_i x + c_i y}{2A}, i = 1, 2, 3$$

- A = area of the triangle
- a_i, b_i, c_i = coefficients computed from the nodal coordinates.

Area of Triangle

The area of the triangle is computed using the determinant formula:

$$A = \frac{1}{2} \begin{vmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{vmatrix} = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

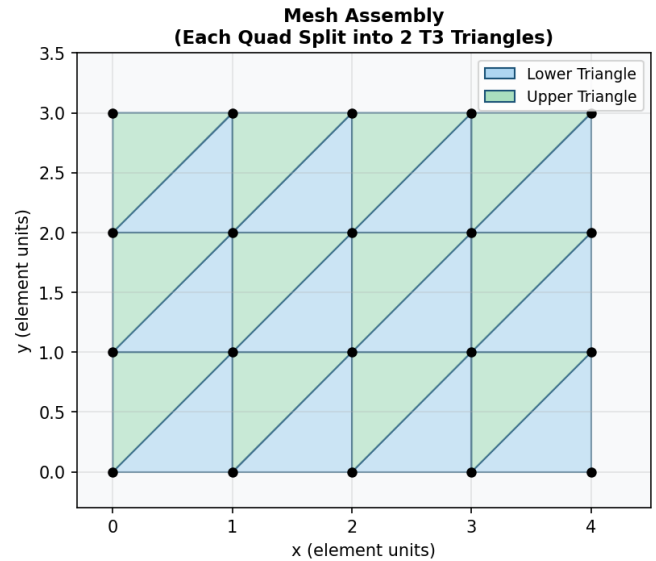
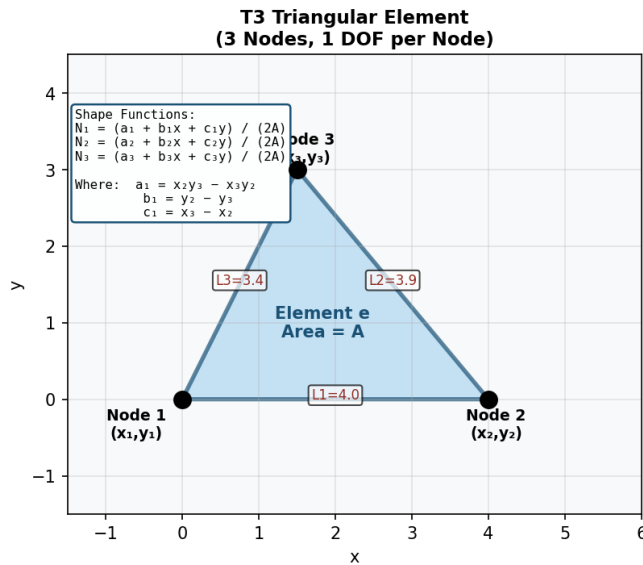
This formula ensures the area is always positive, regardless of node ordering.

Shape Function Coefficients

The coefficients a_i, b_i, c_i are derived from geometry:

$$\begin{aligned} a_1 &= x_2 y_3 - x_3 y_2, & b_1 &= y_2 - y_3, & c_1 &= x_3 - x_2 \\ a_2 &= x_3 y_1 - x_1 y_3, & b_2 &= y_3 - y_1, & c_2 &= x_1 - x_3 \\ a_3 &= x_1 y_2 - x_2 y_1, & b_3 &= y_1 - y_2, & c_3 &= x_2 - x_1 \end{aligned}$$

- These coefficients are directly computed from the node coordinates.
- They make the shape functions linear and satisfy nodal conditions.



Properties of T3 Shape Functions

1. Linearity:

- The temperature varies linearly inside the triangle.
- Gradient of temperature is constant within the element.

2. Partition of Unity:

$$N_1 + N_2 + N_3 = 1 \text{ everywhere inside the triangle}$$

- This ensures interpolation is consistent and no extra temperature is introduced.

3. Nodal Interpolation:

- At node 1: $T = T_1$
- At node 2: $T = T_2$
- At node 3: $T = T_3$

4. Local Support:

- Each N_i is non-zero only within the element.
- Outside the element, $N_i = 0$.

Gradient of Temperature

Since $T(x, y)$ is linear inside the element:

$$\begin{aligned} \frac{\partial T}{\partial x} &= \frac{\partial N_1}{\partial x} T_1 + \frac{\partial N_2}{\partial x} T_2 + \frac{\partial N_3}{\partial x} T_3 \\ \frac{\partial T}{\partial y} &= \frac{\partial N_1}{\partial y} T_1 + \frac{\partial N_2}{\partial y} T_2 + \frac{\partial N_3}{\partial y} T_3 \end{aligned}$$

Gradients are constant across the element, simplifying computation of heat flux.

The [B] Matrix — Gradient of Shape Functions

In 2D heat conduction, the governing equation involves the temperature derivatives $\partial T / \partial x$ and $\partial T / \partial y$.

Since the temperature inside a T3 element is interpolated using linear shape functions:

$$T(x, y) = N_1(x, y)T_1 + N_2(x, y)T_2 + N_3(x, y)T_3$$

the gradient of the temperature is:

$$\begin{bmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_2}{\partial x} & \frac{\partial N_3}{\partial x} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_2}{\partial y} & \frac{\partial N_3}{\partial y} \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix}$$

Gradient of Linear Shape Functions

Recalling the linear shape functions:

$$N_i(x, y) = \frac{a_i + b_i x + c_i y}{2A}, i = 1, 2, 3$$

Taking derivatives:

$$\frac{\partial N_i}{\partial x} = \frac{b_i}{2A}, \quad \frac{\partial N_i}{\partial y} = \frac{c_i}{2A}$$

- b_i, c_i are coefficients determined from the nodal coordinates.
- A is the area of the triangle.
- These derivatives are constant over the element because $N_i(x, y)$ is linear.

Definition of the [B] Matrix

The [B] matrix collects these derivatives into a compact form:

$$[B] = \begin{bmatrix} \frac{\partial N_1}{\partial x} & \frac{\partial N_2}{\partial x} & \frac{\partial N_3}{\partial x} \\ \frac{\partial N_1}{\partial y} & \frac{\partial N_2}{\partial y} & \frac{\partial N_3}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{b_1}{2A} & \frac{b_2}{2A} & \frac{b_3}{2A} \\ \frac{c_1}{2A} & \frac{c_2}{2A} & \frac{c_3}{2A} \end{bmatrix}$$

- First row $\rightarrow$ derivatives w.r.t x
- Second row $\rightarrow$ derivatives w.r.t y

This matrix is analogous to the strain-displacement matrix in structural FEM but here represents temperature gradients.

Temperature Gradient in Vector Form

Using the [B] matrix, the temperature gradient can be written compactly as:

$$\begin{bmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \end{bmatrix} = [B] \begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix} = \begin{bmatrix} \frac{b_1}{2A} & \frac{b_2}{2A} & \frac{b_3}{2A} \\ \frac{c_1}{2A} & \frac{c_2}{2A} & \frac{c_3}{2A} \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix}$$

- This expresses the linear variation of temperature inside the element in terms of nodal temperatures.
- The gradient is constant across the element because the shape functions are linear.

Material Property Matrix [D]

For an isotropic material, the heat flux vector is related to the temperature gradient by Fourier's law:

$$\vec{q} = \begin{bmatrix} q_x \\ q_y \end{bmatrix} = -k \begin{bmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \end{bmatrix} = -k \nabla T$$

Where:

- q_x, q_y = heat fluxes in x and y directions
- $\nabla T = \begin{bmatrix} \partial T / \partial x \\ \partial T / \partial y \end{bmatrix}$ = temperature gradient vector
- k = thermal conductivity of the material

Matrix Formulation

To generalize for FEM calculations, we write Fourier's law in matrix form:

$$\vec{q} = -[D] \nabla T$$

Here, $[D]$ is the material property matrix, also called the thermal conductivity matrix:

$$[D] = \begin{bmatrix} k_{xx} & 0 \\ 0 & k_{yy} \end{bmatrix}$$

- k_{xx} = thermal conductivity in the x-direction
- k_{yy} = thermal conductivity in the y-direction

Isotropic Material Case

For an isotropic material, heat conduction is the same in all directions, so:

$$k_{xx} = k_{yy} = k$$

Then the material property matrix simplifies to:

$$[D] = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$$

- This indicates equal heat conduction along x and y directions.
- In the FEM code, the user enters Thermal Conductivity in the GUI, which sets both k_{xx} and k_{yy} to the same value.

Conduction Stiffness Matrix [Kc]

In the Finite Element Method, the element-level stiffness matrix relates nodal temperatures to heat fluxes.

For steady-state 2D heat conduction, the weak form of the governing equation is:

$$\int_{\Omega^e} (\nabla N)^T k \nabla T \, d\Omega = 0$$

- Ω^e = area of the element
- N = vector of shape functions
- k = thermal conductivity

Using the $[B]$ matrix (gradient of shape functions) and the material property matrix $[D]$, the temperature gradient and heat flux can be expressed as:

$$\nabla T = [B]\{T^e\}, \vec{q} = -[D]\nabla T = -[D][B]\{T^e\}$$

Substituting into the weak form integral gives the element conduction (stiffness) matrix:

$$[K^e] = \int_{\Omega^e} [B]^T [D] [B] \, d\Omega$$

For 2D plane conduction in a thin plate of thickness t (assuming heat flows uniformly across thickness):

$$[K_c] = \int_{\Omega^e} t [B]^T [D] [B] \, d\Omega$$

- t = thickness of the element

For linear T3 elements and isotropic material, both $[B]$ and $[D]$ are constant over **the** element. Then, the integral over the element area simplifies to a multiplication:

$$\int_{\Omega^e} [B]^T [D] [B] \, d\Omega = [B]^T [D] [B] \cdot A$$

Where A = area of the triangle.

Thus, the element conduction matrix becomes:

$$K_c = t \cdot A [B]^T [D][B]$$

Convection Stiffness Matrix and Force Vector

When a boundary edge is exposed to convective heat transfer (Newton's law of cooling), additional contributions must be included in the FEM formulation.

For an edge in contact with a fluid at temperature T_∞ :

$$q_n = h(T_s - T_\infty)$$

- q_n = normal heat flux on the boundary
- h = convective heat transfer coefficient
- T_s = surface temperature on the boundary
- T_∞ = ambient temperature

In the weak form of FEM, the boundary contribution is:

$$\int_{\Gamma_c} N^T q_n d\Gamma = \int_{\Gamma_c} N^T h(T - T_\infty) d\Gamma$$

- Γ_c = convecting boundary
- N = shape function vector for the edge

This produces two contributions:

1. Convection stiffness matrix $\rightarrow$ multiplies nodal temperatures T_i, T_j
2. Convection force vector $\rightarrow$ depends on T_∞

Consistent Convection Stiffness Matrix for an Edge

For a linear edge between nodes i and j of length L :

$$[K_h]_{\text{edge}} = \frac{hLt}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

- t = element thickness
- This is called the consistent edge matrix because it ensures proper weighting of nodal contributions along the edge.
- It is symmetric, 2×2 , and depends linearly on h , L , and t .

Convection Force Vector for an Edge

The **force vector** due to ambient heat transfer is:

$$\{f_h\}_{\text{edge}} = \frac{hT_{\infty}Lt}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- Represents heat added (or removed) from the fluid
- Each node on the edge receives half of the total contribution.

Assembly for a Full Element

A T3 triangular element has 3 edges, any of which may be convective. Denote edge j with length L_j . Then:

$$[K_h]_{\text{element}} = \sum_{j=1}^3 \text{convect_flag}(i, j) \cdot \frac{hL_j}{6t} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}_j$$

$$\{f_h\}_{\text{element}} = \sum_{j=1}^3 \text{convect_flag}(i, j) \cdot \frac{hT_{\infty}L_j}{2t} \begin{bmatrix} 1 \\ 1 \end{bmatrix}_j$$

Element and Global Stiffness Matrix

For each element e , the total stiffness matrix is obtained by combining conduction and convection effects:

$$[K^e] = [K_c^e] + [K_h^e]$$

Where:

- $[K_c^e]$ = conduction stiffness matrix
- $[K_h^e]$ = convection stiffness matrix (only for boundary edges)

Similarly, the element force vector is:

$$\{f^e\} = \{f_c^e\} + \{f_h^e\}$$

- $\{f_c^e\}$ = source-related contribution (often zero for pure conduction)
- $\{f_h^e\}$ = convection contribution

Local-to-Global Mapping (Steering Vector)

Each element has local degrees of freedom (DOFs) corresponding to its nodes.

For a T3 element:

$$\{T^e\} = [T_1 \quad T_2 \quad T_3]^T$$

Let the steering vector (or connectivity array) be:

$$g^e = [g_1 \quad g_2 \quad g_3]$$

- g_i gives the global node number corresponding to local node i

Global Stiffness Matrix Assembly

The global system of equations is:

$$[KK]\{T\} = \{f_g\}$$

Where:

- $[KK]$ = global stiffness matrix
- $\{T\}$ = global nodal temperature vector
- $\{f_g\}$ = global force vector

The assembly process consists of adding each element's contribution into the global matrix using the steering vector:

$$KK(g_i, g_j) += K^e(i, j), \text{ for } i, j = 1, 2, 3$$

or equivalently,

$$[KK] = \sum_{e=1}^{n_e} [A^e]^T [K^e] [A^e]$$

- $[A^e]$ = Boolean assembly matrix mapping local DOFs to global DOFs
- n_e = total number of elements

Global Force Vector Assembly

Similarly, the global force vector is assembled as:

$$f_g(g_i) += f^e(i), \text{ for } i = 1, 2, 3$$

or in compact form:

$$\{f_g\} = \sum_{e=1}^{n_e} [A^e]^T \{f^e\}$$

Applying the Dirichlet Boundary Condition

In the global system:

$$[KK]\{T\} = \{f_g\}$$

For nodes where the temperature is prescribed ($T = T_{\text{boundary}}$, e.g., at $x = 0$), the system of equations must be modified so that these values remain fixed during the solution process.

First, the effect of the known temperatures is transferred to the right-hand side by subtracting their contribution from the force vector corresponding to the free nodes:

$$\{f_g\}_{\text{free}} = \{f_g\}_{\text{free}} - [KK]_{\text{free, fixed}} \cdot \{T\}_{\text{boundary}}$$

Next, the stiffness matrix is adjusted by setting the rows and columns associated with the fixed degrees of freedom to zero. The diagonal entries of these fixed positions are then set to 1, and the corresponding entries in the force vector are assigned the prescribed temperature values:

$$[KK]_{\text{fixed, fixed}} = 1, \quad \{f_g\}_{\text{fixed}} = T_{\text{boundary}}$$

This procedure ensures that the fixed nodes retain exactly the specified temperatures while allowing the solver to compute the unknown temperatures at the remaining nodes consistently.

Solving the Linear System

After assembling the global system and applying the boundary conditions, the final problem reduces to a system of linear equations:

$$[KK] \cdot \{T\} = \{f_g\}$$

This system is solved in MATLAB using the backslash operator, which employs an efficient numerical technique (typically LU decomposition for such systems):

$$\{T\} = [KK] \setminus \{f_g\}$$

The resulting vector $\{T\}$ contains the computed temperatures at all nodes. The temperatures at nodes with prescribed (Dirichlet) boundary conditions remain fixed, as they have already been enforced during the modification of the system.

Mesh Generation Strategy:

The L-shaped domain is constructed using up to four rectangular subdomains. Each rectangular block is discretized into a structured grid of quadrilateral cells. Every quadrilateral cell is further subdivided into four triangular elements (T3 elements) by introducing a node at the cell center and connecting it to the four corner nodes. This approach produces a symmetric and well-conditioned mesh, suitable for finite element analysis.

The mesh connectivity is stored in the array **conodesc**, which contains the indices of the three nodes associated with each triangular element. The array **geom** stores the spatial coordinates (x,y) of all nodes in the mesh.

Mesh generation is performed by the function **T3_mesh_final_term_axisymmetric** which ensures proper connectivity across adjacent blocks. In particular, special care is taken to merge nodes along shared boundaries between blocks, preventing duplication and maintaining continuity of the mesh.

Algorithm:

The overall algorithm of the FEM heat transfer solver can be described through the following sequential steps:

1. Input: Read all user-defined parameters from the GUI, including thermal conductivity k , convection coefficient h , thickness t , number of blocks, and prescribed boundary temperature.
2. Mesh Generation: Call the mesh generation function to create the node coordinate array **geom** and the element connectivity array **conodesc**. Determine the total number of nodes n_{nd} and elements n_{el} .
3. Boundary Identification: Identify and assign boundary conditions. Nodes located at $x = 0$ are assigned prescribed temperatures (Dirichlet boundary). For element edges lying on exposed surfaces, set the convection flag to 1.
4. Material Matrix Formation: Construct the material property matrix:

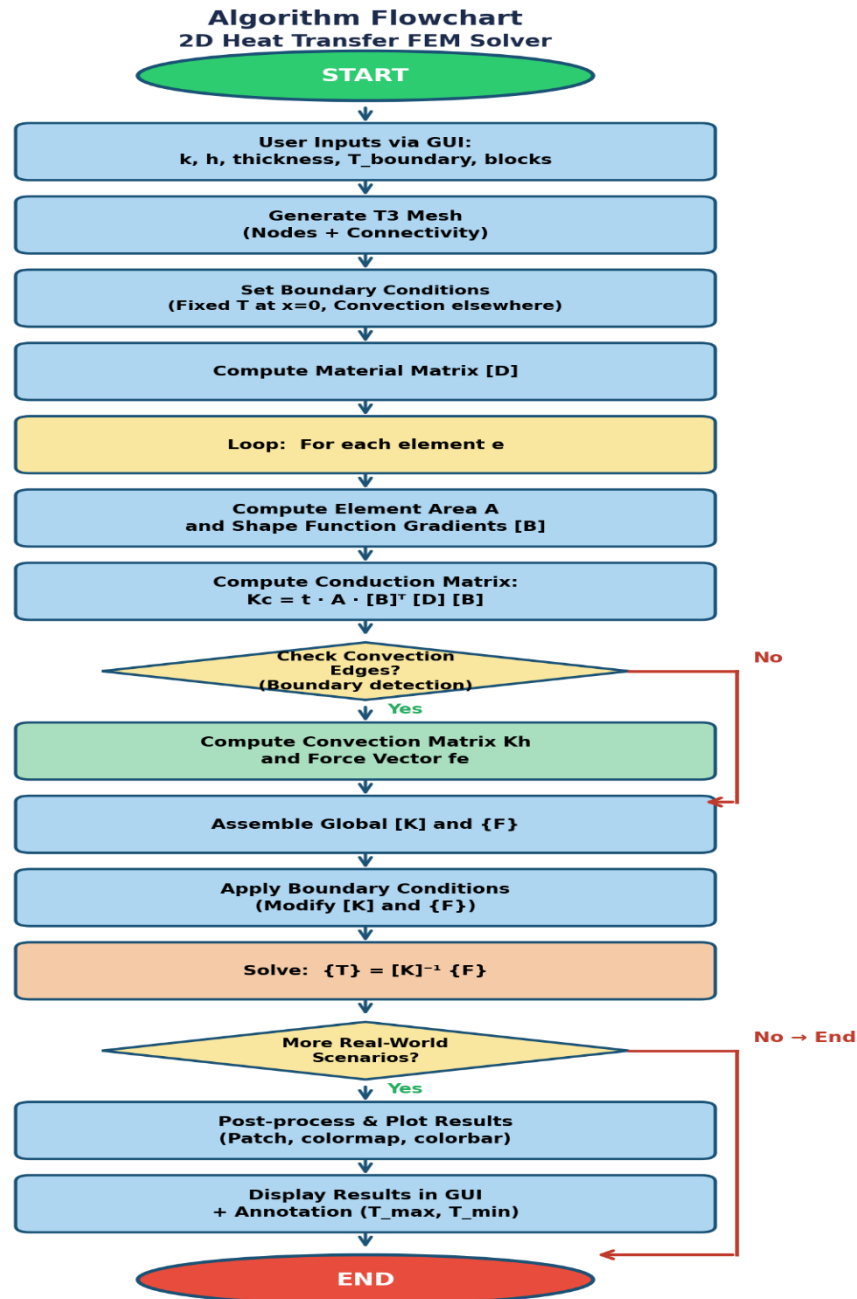
$$[D] = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$$

5. Degree of Freedom (DOF) Numbering: Identify free and fixed degrees of freedom and assign indices to the free DOFs.
6. Element Loop and Assembly: Loop over all elements. For each element, call the function **elem_T3_heat** to compute the element matrices $[B]$, area A , conduction matrix $[K_c]$, convection matrix $[K_h]$, and force vector $\{f_e\}$. Assemble these contributions into the global stiffness matrix $[KK]$ and global force vector $\{f_g\}$.
7. Apply Boundary Conditions: Modify the global system to enforce Dirichlet boundary conditions (prescribed temperature at $x = 0$).
8. Solve the System: Solve the linear system:

$$\{T\} = [KK] \setminus \{f_g\}$$

9. Plot the temperature distribution using a patch plot with color interpolation and display a colorbar.
10. Run predefined real-world scenarios (e.g., CPU cooling or building wall) using the same FEM framework with different input parameters.

Algorithm Flowchart



Pre-Processing:

Pre-processing is the initial phase of the finite element method where the problem is fully defined before any numerical solution is performed. This stage ensures that the solver can run efficiently without additional user intervention. It involves examining the geometry, identifying boundary conditions, generating the mesh, establishing node connectivity, and numbering the degrees of freedom (DOFs).

Geometry and Boundary Conditions for Mesh Generation

The first step is a thorough examination of the problem geometry, boundary conditions, and initial conditions. This information determines how the domain is discretized and how the governing equations will be solved. Two types of boundary conditions are identified:

1. **Fixed Temperature Nodes:** Nodes with prescribed temperatures are identified by checking if their x -coordinate equals zero. For these nodes, the corresponding entry in the nodal load array is set to the boundary temperature.
2. **Convection Edges:** Element edges exposed to convective heat transfer are tagged. A double loop over all elements and their local edges checks geometric criteria. Edges are marked as convecting ($\text{convect}(i,j) = 1$) if they lie on surfaces such as:
 - $x = 70$ (right vertical face)
 - $y = 0$ or $y = 70$ (outer horizontal faces)
 - $y = 10, 20, \dots, 60$ with $x \geq 10$ (inner horizontal step faces)
 - $x = 10$ with y in non-blocked regions

The mesh is generated using the function **T3_mesh_final_term_axisymmetric**, which processes the domain block by block:

- Each rectangular block is divided into **NoElemInXdirec**(b) $\times$ **NoElemInYdirec**(b) quadrilateral cells.
- Each quadrilateral cell is further split into 4 triangular T3 elements by introducing a center node at the midpoint of the cell and connecting it to all four corners.
- For the first block, node indices are assigned sequentially based on loop counters.
- For subsequent blocks, left-edge nodes are matched with existing coordinates in **geom** to avoid duplicate nodes at block interfaces.

The output is:

- **geom** — an $n_{nd} \times 2$ matrix containing all node coordinates (x, y)
- **conodesc** — an $n_{el} \times 3$ matrix listing the 3 node indices per triangular element

This approach produces a symmetric, well-conditioned mesh suitable for finite element analysis.

Node Connectivity

All node-to-element connections are organized in a tabular format. For quadrilateral meshes (or subdivided T3 triangles), each element is associated with its local nodes in a consistent order (typically anti-clockwise). For example:

- Element 1 connects nodes 1, 2, 9, 8
- Element 2 connects nodes 2, 3, 10, 9

This structure simplifies the assembly of element matrices into the global stiffness matrix. After identifying boundary conditions, free degrees of freedom are numbered sequentially to map local element DOFs to the global system. An array $\mathbf{nf}(\mathbf{i}, \mathbf{j})$ is initialized for all nodes. Free DOFs are assigned integers $1, 2, 3, \dots, n_{\text{total}}$, which determine the positions of rows and columns in the global stiffness matrix $[KK]$.

Processing:

The processing stage represents the core of the Finite Element Method (FEM), where the theoretical formulation of heat conduction is systematically translated into computational steps to determine the temperature distribution across the domain. Within each triangular element (T3), the temperature field is approximated using linear shape functions, which ensure a continuous and smooth interpolation between nodal temperatures. For nodes located at $(x_1, y_1), (x_2, y_2), (x_3, y_3)$, the temperature at any point (x, y) within the element is expressed as:

$$T(x, y) = N_1(x, y)T_1 + N_2(x, y)T_2 + N_3(x, y)T_3$$

where $N_i(x, y)$ are the linear shape functions satisfying the condition $N_1 + N_2 + N_3 = 1$. This property ensures linear interpolation of temperature within each element, preserving both continuity and consistency across the mesh.

To compute the element matrices, the weak form of the 2D steady-state heat conduction equation is employed. The element conduction stiffness matrix is calculated as:

$$K_c = t A [B]^T [D] [B]$$

Here, $[B]$ is the strain-displacement-like matrix containing the gradients of the shape functions, $[D] = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$ is the material property matrix for isotropic conduction, t is the element thickness, and A is the area of the triangular element, derived directly from the nodal coordinates.

Elements adjacent to convective boundaries contribute additional terms to account for heat exchange with the ambient environment. For an edge of length L exposed to convection, the edge contributions to the stiffness matrix and force vector are:

$$k_h^{edge} = \frac{hLt}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad f_{hedge} = 2hT_\infty L_t \begin{Bmatrix} 1 \\ 1 \end{Bmatrix}$$

where h is the convective heat transfer coefficient and T_∞ is the ambient temperature. These contributions are superimposed on the element conduction matrix and force vector, resulting in the total element stiffness matrix $K_e = K_c + K_h$ and element force vector f_e .

The global system is assembled by mapping each element's local matrices to the global stiffness matrix $[KK]$ and force vector $\{f_g\}$ using a steering vector, which links the local node numbers to the corresponding global degrees of freedom. This assembly procedure ensures that all element interactions are accurately captured across the entire mesh.

Dirichlet boundary conditions (prescribed temperatures) are incorporated by appropriately modifying the global matrices: the global force vector is adjusted to account for the influence of fixed nodes, and the stiffness matrix is modified so that the known temperatures are exactly enforced at the boundaries.

The final system of equations,

$$[KK]\{T\} = \{f_g\},$$

is solved in MATLAB using the backslash operator ($\backslash$), which applies an efficient direct solver (sparse LU factorization for large systems or dense solvers for smaller systems). The computed solution vector contains the temperatures at all free nodes, with the fixed node temperatures already embedded. This method provides a precise and reliable representation of both conduction and convection phenomena within the domain, accurately capturing the thermal behavior of the system.

Post-Processing:

The obtained data and results will be compared with existing findings for similar boundary conditions and problems. This comparison will carefully consider slight variations in the model geometry and meshing approach, as differences in domain discretization and element topology can introduce minor deviations from reference solutions. Despite these variations, the overall trends and numerical values are expected to remain in close agreement with previously published results, thereby validating the accuracy and reliability of the implemented finite element formulation.

Results (sample Input Output):

Sample input 1:

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity: 500

Heat Convection Coefficient: 1

Show Plot:

Number of Blocks (Enter between 1 to 4): 4

Thickness(in mm): 10

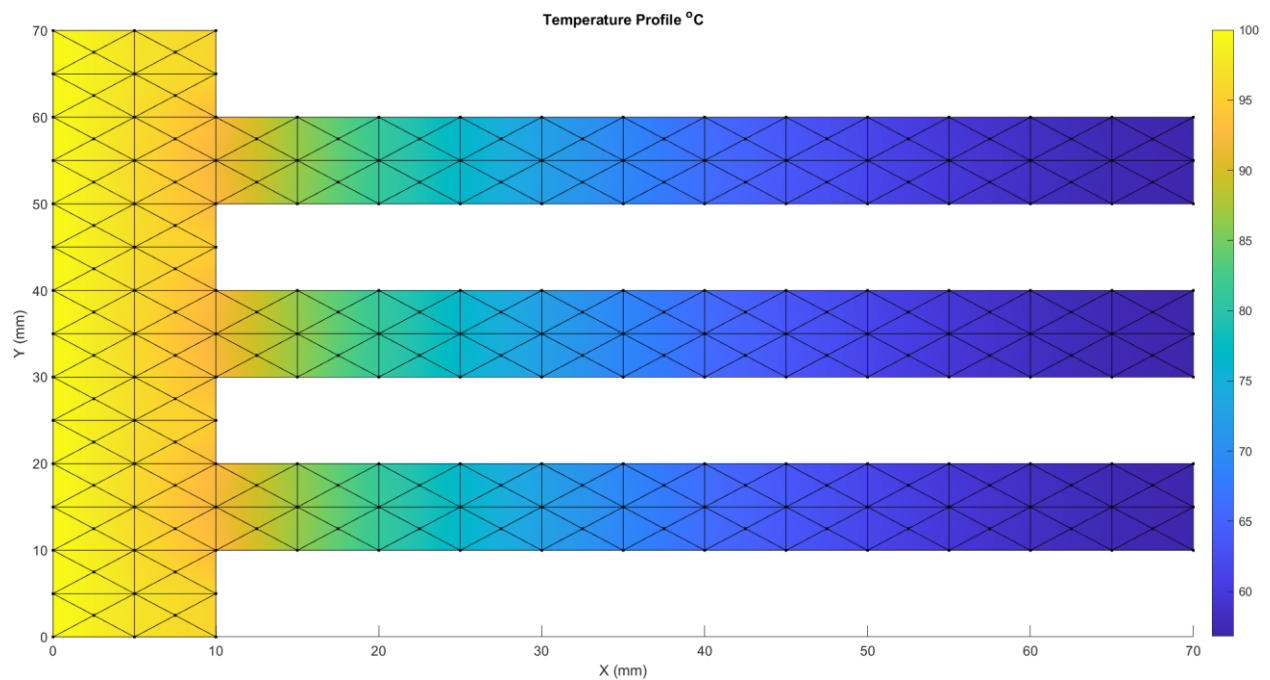
Bounded Temperature (Initial): 100

Real-World Heat Transfer Applications

Select Scenario: Electronic Device Cooling ▼ **Run Application**

Results will appear here after running a scenario.

Sample output 1:



Sample input 2: (when thermal conductivity decreases)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity:

Heat Convection Coefficient:

Show Plot:

Number of Blocks (Enter between 1 to 4):

Thickness(in mm):

Bounded Temperature (Initial):

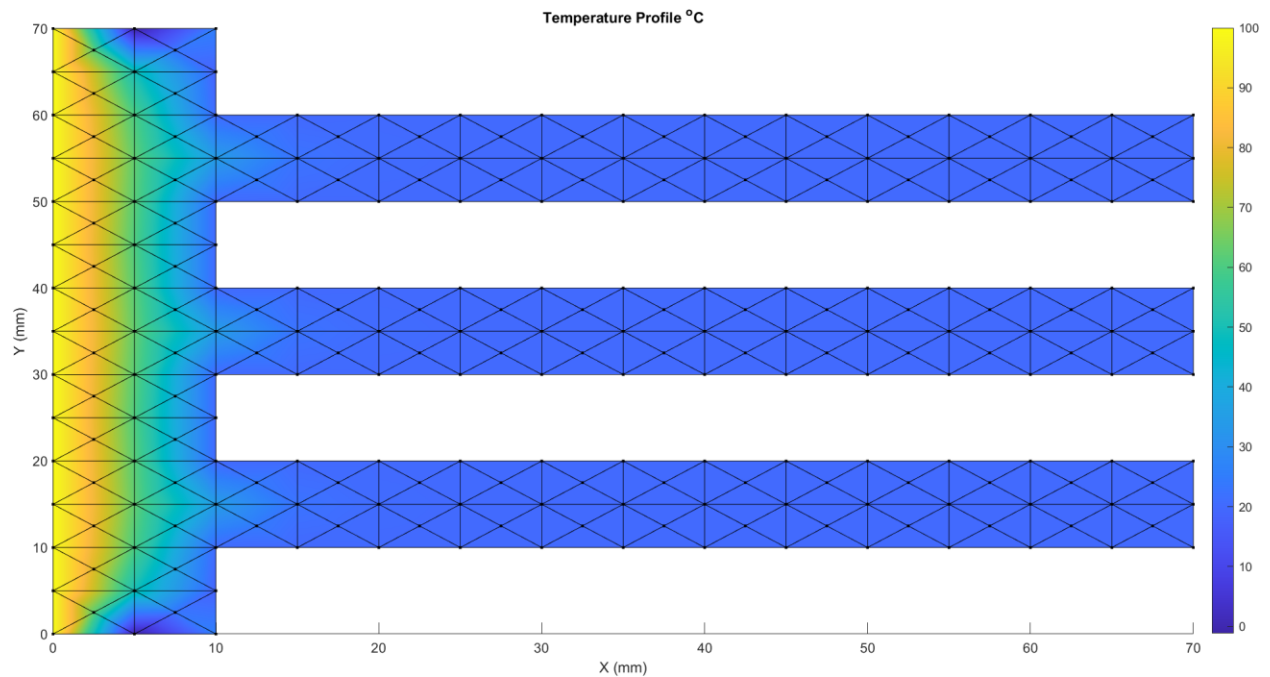
Real-World Heat Transfer Applications

Select Scenario:

Run Application

Results will appear here after running a scenario.

Sample output 2:



Sample input 3: (when thermal conductivity increases)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity: 1000

Heat Convection Coefficient: 10

Show Plot:

Thickness(in mm): 10

Number of Blocks (Enter between 1 to 4): 3

Bounded Temperature (Initial): 1000

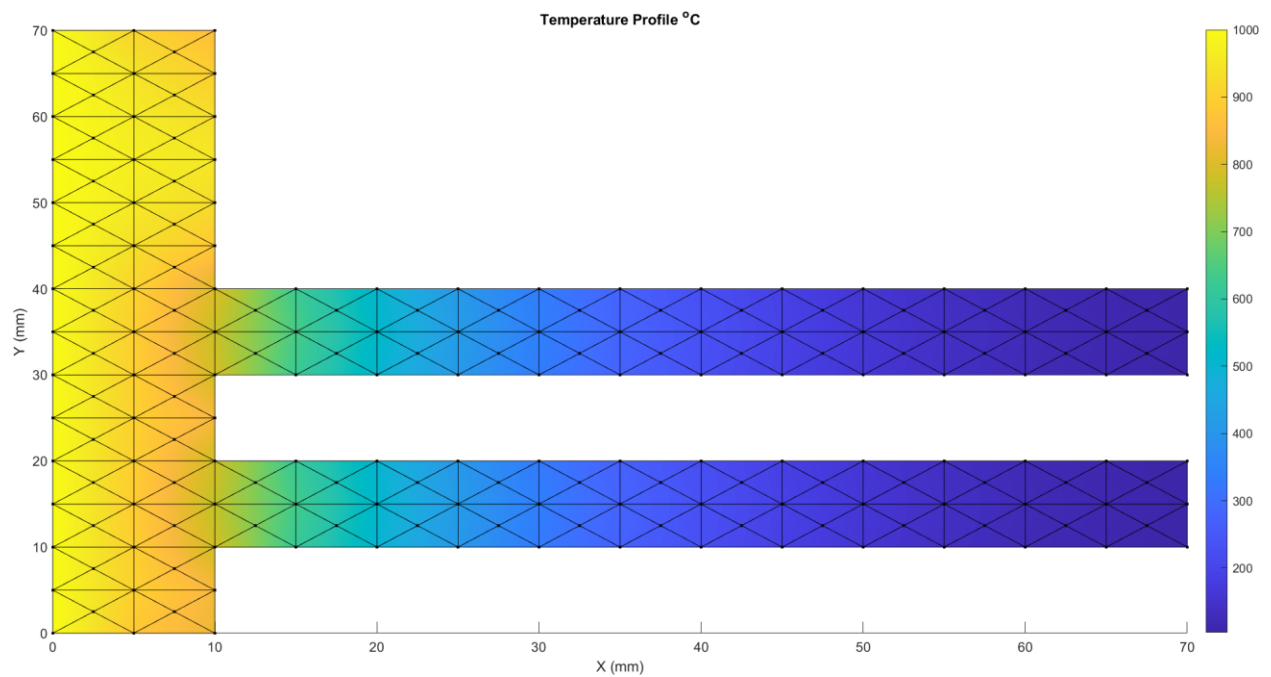
Real-World Heat Transfer Applications

Select Scenario: Electronic Device Cooling

Run Application

Results will appear here after running a scenario.

Sample output 3:



Sample input 4: (when g and k decreases)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity: 0.001

Heat Convection Coefficient: 0.0001

Show Plot:

Number of Blocks (Enter between 1 to 4): 4

Thickness(in mm): 10

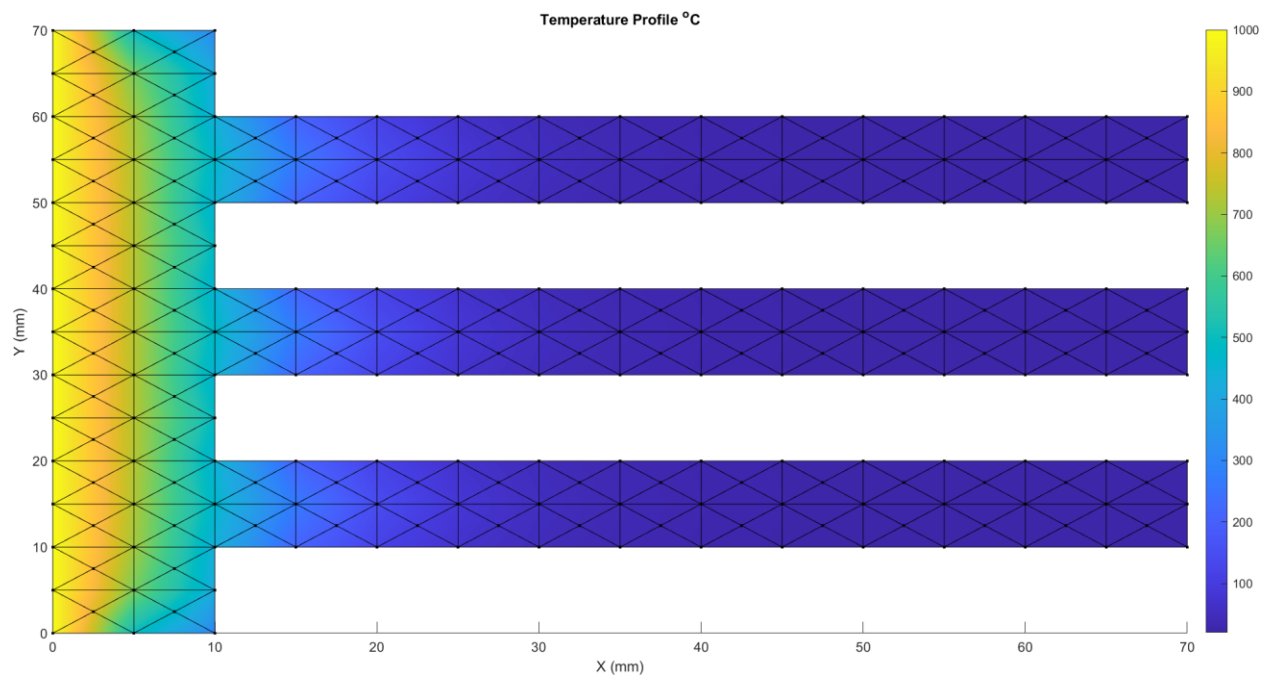
Bounded Temperature (Initial): 1000

Real-World Heat Transfer Applications

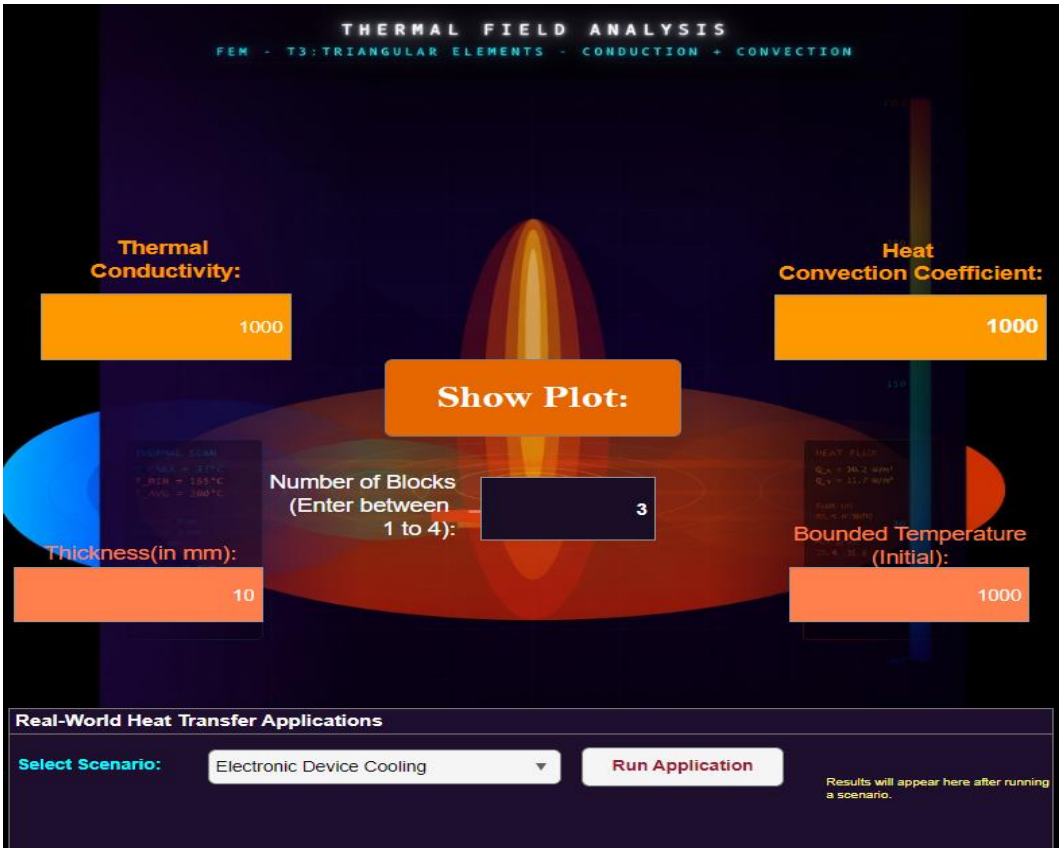
Select Scenario: Electronic Device Cooling ▼ **Run Application**

Results will appear here after running a scenario.

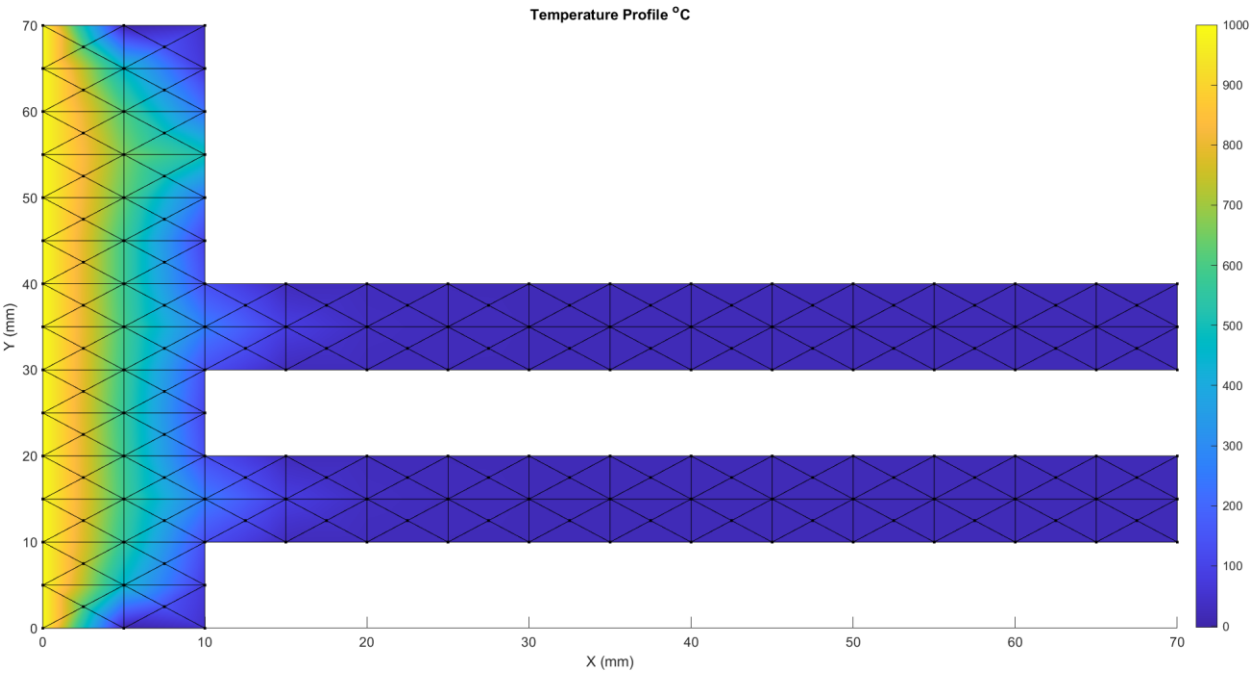
Sample output 4:



Sample input 5:(when g and k increases)



Sample output 5:



Sample input 6: (when thickness increases)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity: 0.001

Heat Convection Coefficient: 0.0001

Show Plot:

Number of Blocks (Enter between 1 to 4): 4

Thickness(in mm): 1e+05

Bounded Temperature (Initial): 1000

Real-World Heat Transfer Applications

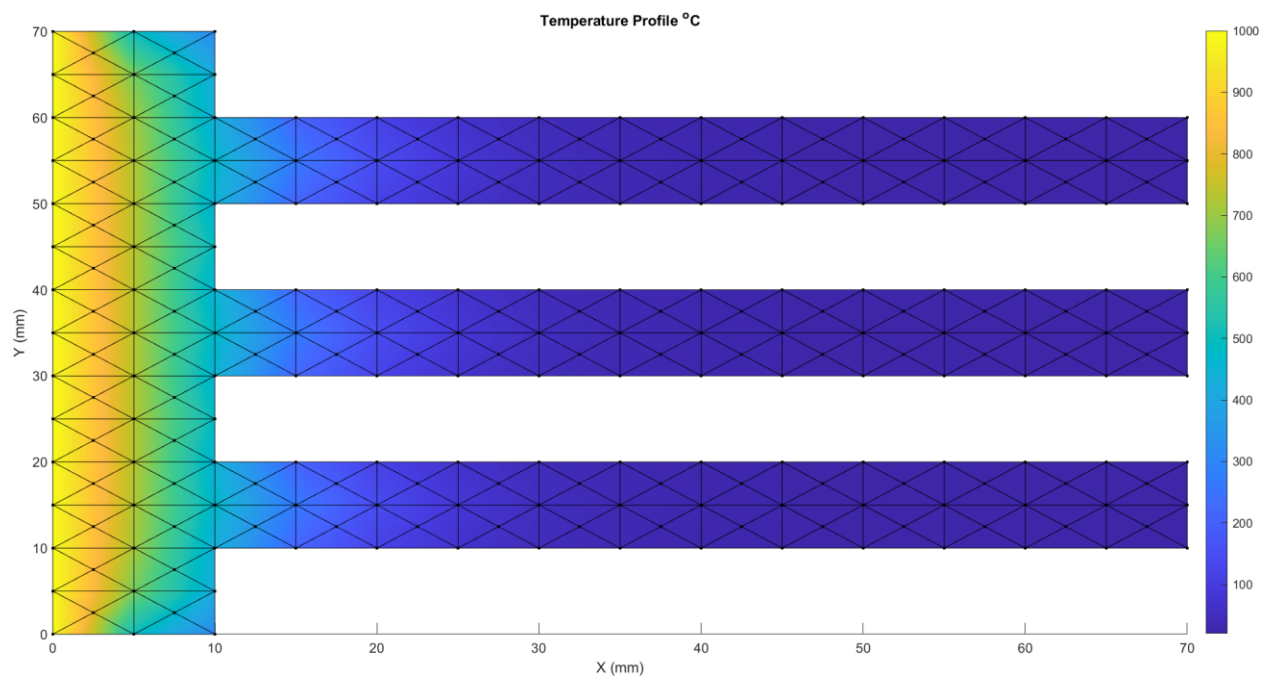
Select Scenario: Electronic Device Cooling ▼ **Run Application**

Results will appear here after running a scenario.

THERMAL SCAN
T_MAX = 155°C
T_MIN = 155°C
T_AVG = 150°C
TIME = 1000s

HEAT FLUX
Q_x = 30.2 W/m²
Q_y = 11.7 W/m²
Q_z = 0 W/m²
Q_TOT = 32.5 W/m²

Sample output 6:



Sample input 7: (when thickness decreases)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity: 0.001

Heat Convection Coefficient: 0.0001

Show Plot:

Number of Blocks (Enter between 1 to 4): 4

Thickness(in mm): 1e-05

Bounded Temperature (Initial): 1000

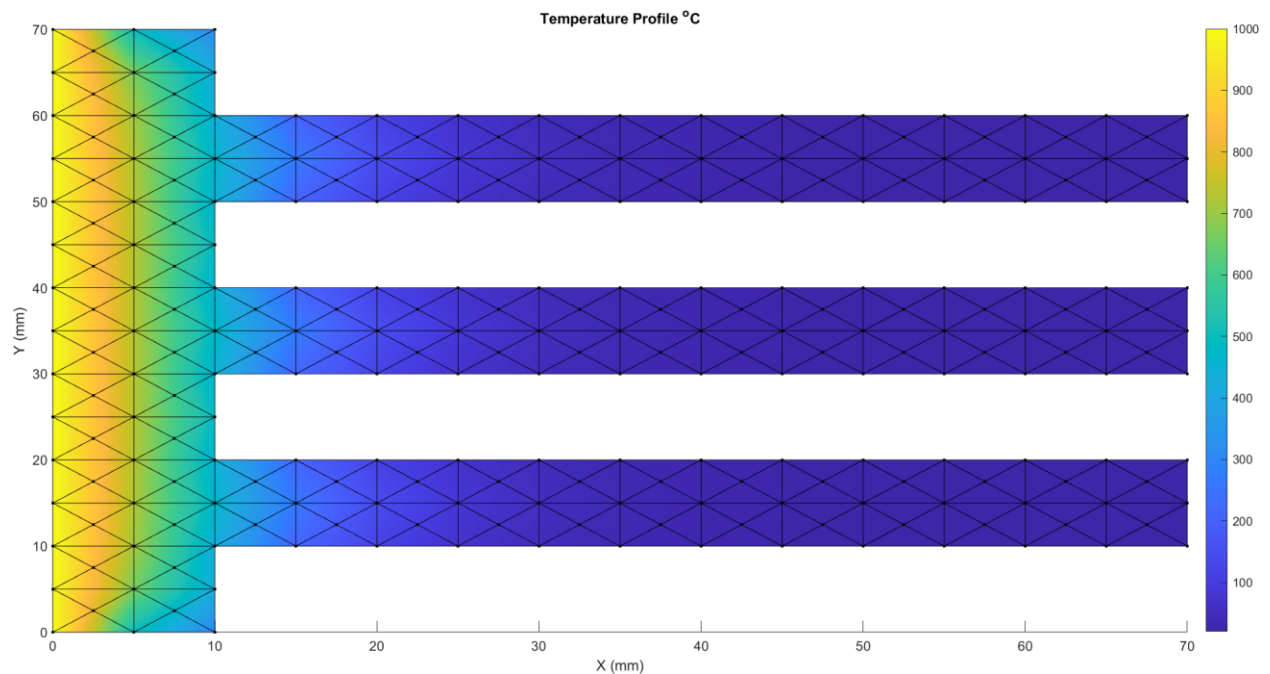
Real-World Heat Transfer Applications

Select Scenario: Electronic Device Cooling

Run Application

Results will appear here after running a scenario.

Sample output 7:



Sample input 8: (when initial bounded temp changes)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity:

Heat Convection Coefficient:

Show Plot:

Number of Blocks (Enter between 1 to 4):

Thickness(in mm):

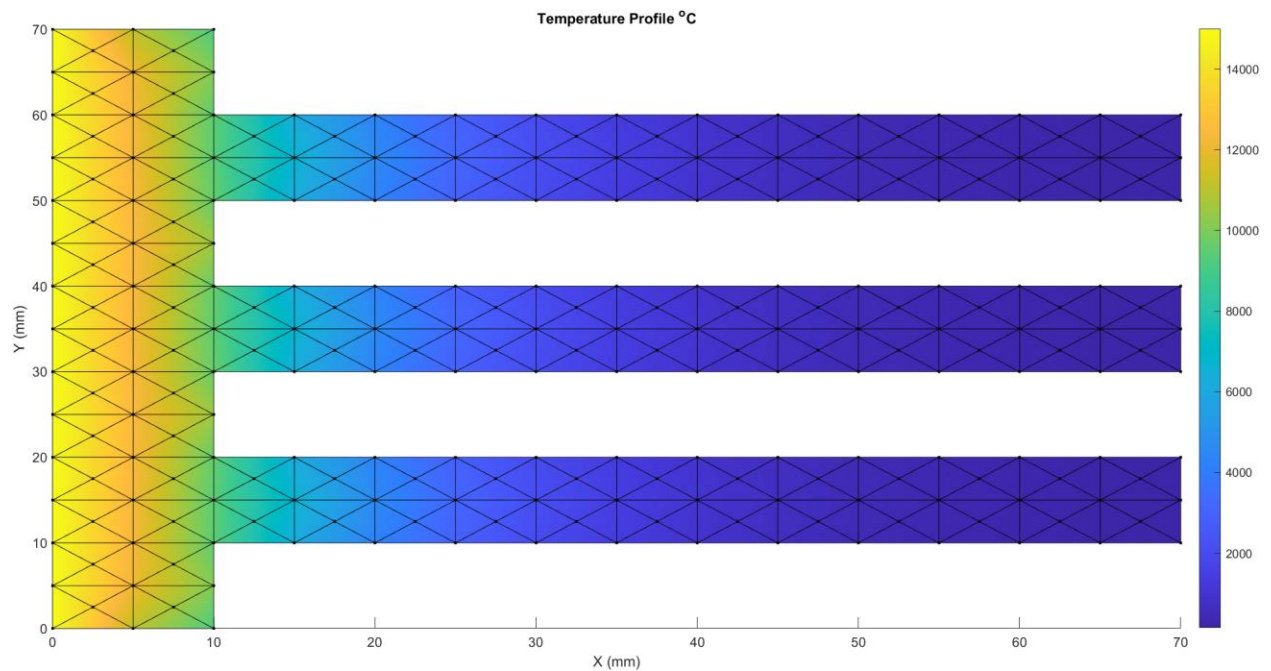
Bounded Temperature (Initial):

Real-World Heat Transfer Applications

Select Scenario: **Run Application**

Results will appear here after running a scenario.

Sample output 8:



Sample input 9: (when block changes)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity:

Heat Convection Coefficient:

Show Plot:

Number of Blocks (Enter between 1 to 4):

Thickness(in mm):

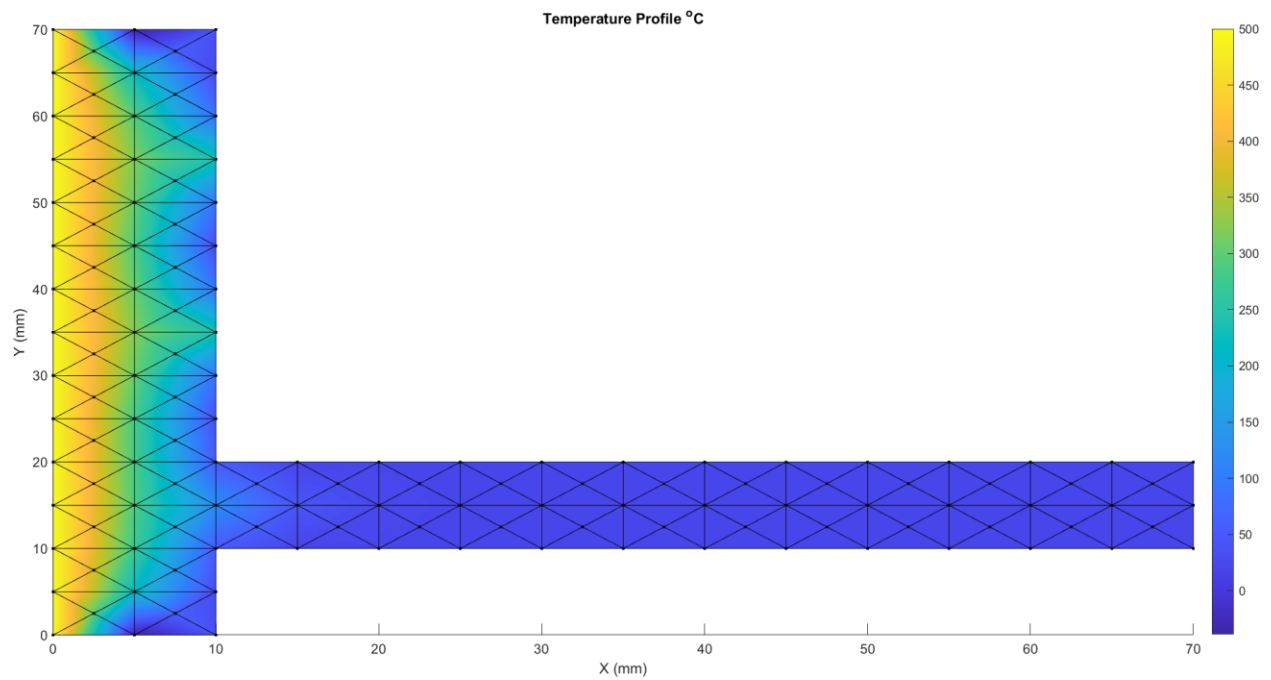
Bounded Temperature (Initial):

Real-World Heat Transfer Applications

Select Scenario: **Run Application**

Results will appear here after running a scenario.

Sample output 9:



Sample input 10: (when block changes)

THERMAL FIELD ANALYSIS
FEM - T3: TRIANGULAR ELEMENTS - CONDUCTION + CONVECTION

Thermal Conductivity:

Heat Convection Coefficient:

Show Plot:

Number of Blocks (Enter between 1 to 4):

Thickness(in mm):

Bounded Temperature (Initial):

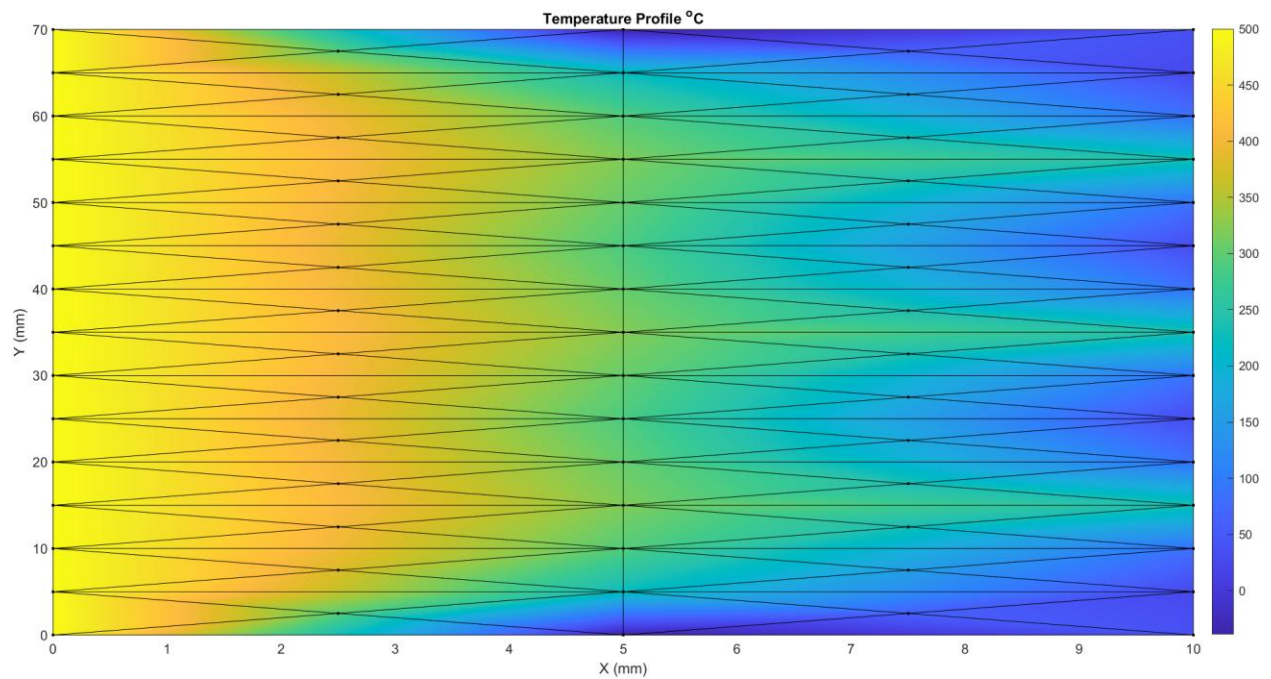
Real-World Heat Transfer Applications

Select Scenario: **Run Application**

Results will appear here after running a scenario.



Sample output:



Real world heat transfer application:

Case-01: Electronic Device Cooling

Real-World Heat Transfer Applications

Select Scenario:

Electronic Device Cooling

Run Application

Electronic Device Cooling

Material : Silicon

$k = 148 \text{ W/(m}\cdot\text{K)}$

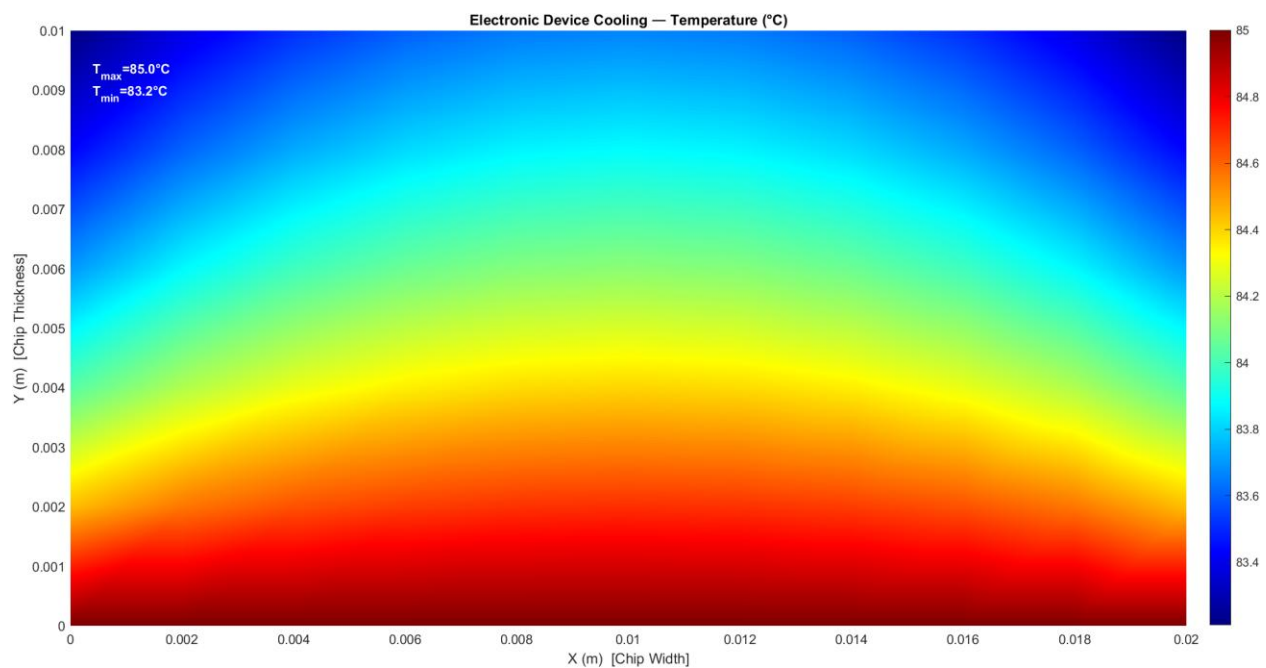
$h = 250 \text{ W/(m}^2\cdot\text{K)}$

$T_{\text{base}} = 85 \text{ }^{\circ}\text{C}$ | $T_{\text{ambient}} = 25 \text{ }^{\circ}\text{C}$

Domain: 20 mm $\times$ 10 mm

→ See Figure 2 for temperature map.

Output:



Case-02: Building Wall Heating

Real-World Heat Transfer Applications

Select Scenario:

Building Wall Heating

Run Application

Building Wall Heating

Material : Concrete

$k = 1.7 \text{ W/(m}\cdot\text{K)}$

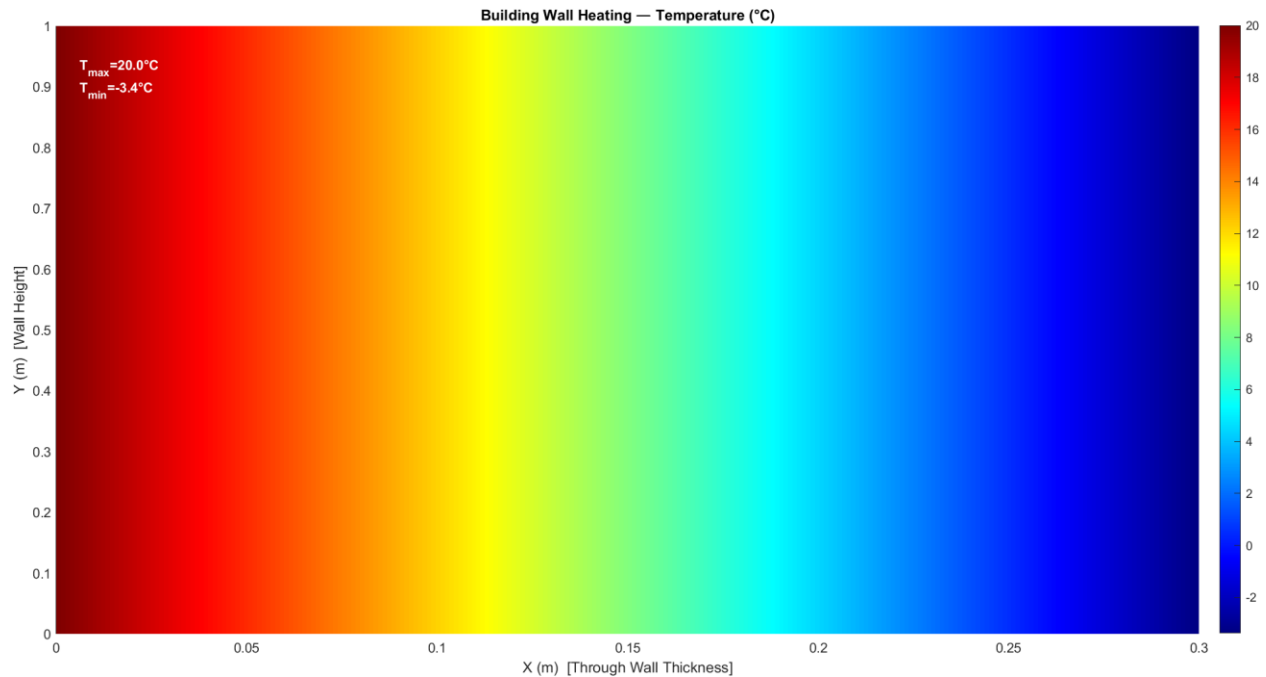
$h_{\text{ext}} = 20 \text{ W/(m}^2\cdot\text{K)}$

$T_{\text{interior}} = 20 \text{ }^{\circ}\text{C}$ | $T_{\text{exterior}} = -10 \text{ }^{\circ}\text{C}$

Wall: 300 mm thick $\times$ 1000 mm tall

→ See Figure 3 for temperature map.

Output:



Code with GUI:

```
classdef EEE_PROJECT_FROM_95_AND_96 < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                matlab.ui.Figure
        ShowPlotButton          matlab.ui.control.Button
        BoundedTemperatureInitialEditField matlab.ui.control.NumericEditField
        BoundedTemperatureInitialLabel matlab.ui.control.Label
        ThicknessinmmEditField  matlab.ui.control.NumericEditField
        ThicknessinmmEditFieldLabel matlab.ui.control.Label
        NumberOfBlocksEnterbetween1to4EditField matlab.ui.control.NumericEditField
        NumberOfBlocksEnterbetween1to4Label matlab.ui.control.Label
        HeatConvectionCoefficientEditField matlab.ui.control.NumericEditField
        HeatConvectionCoefficientLabel matlab.ui.control.Label
        ThermalConductivityEditField matlab.ui.control.NumericEditField
        ThermalConductivityLabel matlab.ui.control.Label
        Image                   matlab.ui.control.Image

        % NEW: Real-World Application UI Components
        RealWorldPanel          matlab.ui.container.Panel
        AppScenarioLabel         matlab.ui.control.Label
        AppScenarioDropDown      matlab.ui.control.DropDown
        RunRealWorldButton       matlab.ui.control.Button
        RealWorldResultsLabel    matlab.ui.control.Label
    end
```

```

% Callbacks that handle component events
methods (Access = private)

    % Button pushed function: ShowPlotButton
    function ShowPlotButtonPushed(app, event)

clear global
global nnd nel nodes DOF elDOF n Block
global geom conodesc convect nf Nodal_loads
global Length Width NoElemInXdirec NoElemInYdirec ElemSizeInXdirec ElemSizeInYdirec
X_origin Y_origin
format long g

%% Inputs
% DOF
    nodes = 3;           % Number of nodes each element
    DOF = 1;             % Number of degrees of freedom
    elDOF = nodes*DOF;   % Element degrees of freedom
% Material
    ThermalConductivity = app.ThermalConductivityEditField.Value;
%Coefficient of thermal conductivity, Btu/(h-ft-F)
    Kyy = ThermalConductivity;
    h = app.HeatConvectionCoefficientEditField.Value;           %Heat convection
    coefficient, Btu/(h-ft-F)
    thick = app.ThicknessinmmEditField.Value;                   %Beam thickness in mm
% Geometries
    Block = app.NumberofBlocksEnterbetween1to4EditField.Value; % Number of
    rectangular block in the model
    a = 70/7;           % Constant geometry parameter
    Length = [a 60 60 60]; % Length of blocks
    Width = [70 a a a]; % Width of blocks
    NoElemInAparam = 2; % Number of elements in each "a" parameter
    NoElemInXdirec = [1 6 6 6]*NoElemInAparam;% Number of elements in the x direction
of each block
    NoElemInYdirec = [7 1 1 1]*NoElemInAparam;% Number of elements in the y direction
of each block
    ElemSizeInXdirec = Length./NoElemInXdirec; % Element size in the x
direction
    ElemSizeInYdirec = Width./NoElemInYdirec; % Element size in the x direction
    X_origin = [0 10 10 10]; % X origin of the global coordinate system
    Y_origin = [0 10 30 50]; % Y origin of the global coordinate system
    T3_mesh_final_term_axisymmetric

% Boundaries & Loading
    % Nodes subjected to given temperature source
    InitialBoundedTemp = app.BoundedTemperatureInitialEditField.Value; % *****F
    nf = ones(nnd,DOF);
    Nodal_loads= zeros(nnd,DOF);
    for i=1:nnd
        % Find nodes at x = 0
        if geom(i,1) == 0
            Nodal_loads(i,1) = InitialBoundedTemp;
        end
    end
end

```

```

% Element edges subjected to convection of free-steam temperature
T_inf = 20; % F
convect = zeros(nel,nodes);
k = [2 3 1];
for i=1:nel
    for j=1:nodes
        % Find x = 70
        if all(geom(conodesc(i,[j k(j)]),1) == 70)
            convect(i,j) = 1;
        end

        % Find y = long block top/bottom & x >= 10
        find_y = [0 70];
        for p = 1:length(find_y)
            isConvect1(p) = all(geom(conodesc(i,[j k(j)]),2) == find_y(p));
        end
        if any(isConvect1) && all(geom(conodesc(i,[j k(j)]),1) >= 0)
            convect(i,j) = 1;
        end

        % Find y = long block top/bottom & x >= 10
        find_y = [10 20 30 40 50 60];
        for p = 1:length(find_y)
            isConvect2(p) = all(geom(conodesc(i,[j k(j)]),2) == find_y(p));
        end
        if any(isConvect2) && all(geom(conodesc(i,[j k(j)]),1) >= 10)
            convect(i,j) = 1;
        end

        % Find x = 10 & y is not inside block
        find_y = [0 10; 20 30; 40 50; 60 70];
        for p = 1:4
            isConvect3(p) = all([geom(conodesc(i,[j k(j)]),2) >= find_y(p,1);
                                geom(conodesc(i,[j k(j)]),2) <= find_y(p,2)]);
        end
        if any(isConvect3) && all(geom(conodesc(i,[j k(j)]),1) == 10)
            convect(i,j) = 1;
        end
    end
end
end

```

```

%% Pre-processing
% Form the material property matrix
dee = [ThermalConductivity    0 ;
       0    Kyy];

```

```

% Counting of the free degrees of freedom
n=0;
for i=1:nnd
    for j=1:Dof
        if nf(i,j) ~= 0
            n=n+1;
            nf(i,j)=n;
        end
    end
end

```



```

end
end

%% Matrix assembly
% Assemble the global stiffness matrix and convection force vector
kk = zeros(n,n); %
fg = zeros(n,DOF);
for i=1:nel
    [bee,cee,fee,g,A] = elem_T3_heat(i); % Form stiffness matrix(bee), convection
vector (cee)
                                % element force function(fee), and
                                % steering vector(g)

    % Stiffness matrix
    kc = thick*A*bee'*dee*bee;      % Compute conduction matrix
    kh = thick*h*cee;               % Compute convection matrix
    ke = kc + kh;                   % Compute local stiffness matrix
    kk = form_kk(kk,ke,g);          % Assemble global stiffness matrix

    % Force vector
    fe = thick*h*T_inf*fee;
    fg = form_fg(fg,fe,g);
end

% Modify the global force vector due to boundaries
idT = find(Nodal_loads~=0);
for i=1:nnd
    if ~any(idT==i)
        fg(i) = fg(i) - sum(kk(i,idT)*Nodal_loads(idT));
    end
end
for i=1:nnd
    if Nodal_loads(i) ~= 0
        fg(i) = Nodal_loads(i); % Given temperature source
    end
end
[kk(idT,:),kk(:,idT)] = deal(0);
for i=1:length(idT)
    kk(idT(i),idT(i)) = deal(1);
end

%% Solve Problem
tg = kk\fg ; % solve for unknown displacements

figure(1)
patch('Faces', conodesc, 'Vertices', geom, 'FaceVertexCData',tg, ...
'Facecolor','interp','Marker','.')
colorbar;
set(gcf,'position',[400 500 500 400])
title('Temperature Profile ^{o}C')
xlabel('X (mm)')
ylabel('Y (mm)')

```

```

function T3_mesh_final_term_axisymmetric
%global nnd NoElemInXdirec NoElemInYdirec ElemSizeInXdirec ElemSizeInYdirec X_origin
Y_origin geom conodesc nel Block
    k = 0;
    for b = 1:Block
        for i = 1:NoElemInXdirec(b)
            for j=1:NoElemInYdirec(b)
                k = k + 1;
                if b == 1
                    n1 = j + (i-1)*(2*NoElemInYdirec(b)+1);
                    n2 = j + i*(2*NoElemInYdirec(b)+1);
                    n3 = n1 + 1;
                    n4 = n2 + 1;
                    n5 = j + (2*i-1)*NoElemInYdirec(b)+i;
                    geom(n1,:) = [(i-1)*ElemSizeInXdirec(b)+X_origin(b) (j-
1)*ElemSizeInYdirec(b)+Y_origin(b)];
                    geom(n3,:) = [(i-1)*ElemSizeInXdirec(b)+X_origin(b)
j*ElemSizeInYdirec(b)+Y_origin(b)    ];
                else
                    if i == 1
                        n1 = find(and(geom(:,1)==X_origin(b), ...
round(geom(:,2),2)==round(Y_origin(b)+(j-
1)*ElemSizeInYdirec(b),2)));
                        n2 = nnd + j + NoElemInYdirec(b);
                        n3 = n1 + 1;
                        n4 = n2 + 1;
                        n5 = nnd + j;
                    else
                        n1 = nnd + j + (i-2)*(2*NoElemInYdirec(b)+1) +
NoElemInYdirec(b);
                        n2 = nnd + j + (i-1)*(2*NoElemInYdirec(b)+1) +
NoElemInYdirec(b);
                        n3 = n1 + 1;
                        n4 = n2 + 1;
                        n5 = nnd + j + (2*i-3)*NoElemInYdirec(b) + i-1 +
NoElemInYdirec(b);
                    end
                    end
                    nnd_b(b) = n4;
                    geom(n2,:) = [i*ElemSizeInXdirec(b)+X_origin(b)      (j-
1)*ElemSizeInYdirec(b)+Y_origin(b)];
                    geom(n4,:) = [i*ElemSizeInXdirec(b)+X_origin(b)
j*ElemSizeInYdirec(b)+Y_origin(b)    ];
                    geom(n5,:) = mean([geom(n1,:); geom(n4,:)]);
                    nel = 4*k;
                    m = nel -3;
                    conodesc(m,:) = [n1 n2 n5];
                    m = nel -2;
                    conodesc(m,:) = [n2 n4 n5];
                    m = nel -1;
                    conodesc(m,:) = [n4 n3 n5];
                    conodesc(nel,:) = [n3 n1 n5];
                end
            end
        end
        nnd = n4;
    end
end

```

```

end
end

% This function returns the coordinates of the nodes of element i
% and its steering vector
function [bee,cee,fee,g,A] = elem_T3_heat(i)
%global nodes DOF geom conodesc nf convect

    x1 = geom(conodesc(i,1),1); y1 = geom(conodesc(i,1),2);
    x2 = geom(conodesc(i,2),1); y2 = geom(conodesc(i,2),2);
    x3 = geom(conodesc(i,3),1); y3 = geom(conodesc(i,3),2);

    A = (0.5)*det([1 x1 y1; ...
                  1 x2 y2; ...
                  1 x3 y3]);

    L = [((x2-x1)^2 + (y2-y1)^2)^.5 ...
         ((x3-x2)^2 + (y3-y2)^2)^.5 ...
         ((x1-x3)^2 + (y1-y3)^2)^.5];

    m11 = (x2*y3 - x3*y2)/(2*A);
    m21 = (x3*y1 - x1*y3)/(2*A);
    m31 = (x1*y2 - y1*x2)/(2*A);
    m12 = (y2 - y3)/(2*A);
    m22 = (y3 - y1)/(2*A);
    m32 = (y1 - y2)/(2*A);
    m13 = (x3 - x2)/(2*A);
    m23 = (x1 - x3)/(2*A);
    m33 = (x2 - x1)/(2*A);

    % Conduction stiffness matrix
    bee = [m12 m22 m32;
           m13 m23 m33];

    % Convection stiffness matrix
    cee = 1/6*(convect(i,1)*L(1)*[2 1 0; 1 2 0; 0 0 0] + ...
             convect(i,2)*L(2)*[0 0 0; 0 2 1; 0 1 2] + ...
             convect(i,3)*L(3)*[2 0 1; 0 0 0; 1 0 2]);

    % Heat force vector
    fee = 1/2*(convect(i,1)*L(1)*[1 1 0]' + ...
              convect(i,2)*L(2)*[0 1 1]' + ...
              convect(i,3)*L(3)*[1 0 1]');

    % Steering vector
    l=0;
    for k=1:nodes
        for j=1:DOF
            l=l+1;
            g(l)=nf(conodesc(i,k),j);
        end
    end
end

% This function assembles the global stiffness matrix

```

```

function kk = form_kk(kk,ke,g)
%global eIDOF
    for i=1:eIDOF
        if g(i) ~= 0
            for j=1:eIDOF
                if g(j) ~= 0
                    kk(g(i),g(j))= kk(g(i),g(j)) + ke(i,j);
                end
            end
        end
    end
end

% This function assembles the global force vector
function fg = form_fg(fg,fe,g)
%global eIDOF
    for i=1:eIDOF
        if g(i) ~= 0
            fg(g(i))= fg(g(i)) + fe(i);
        end
    end
end

% NEW: Real-World Heat Transfer Application Button Callback
function RunRealWorldButtonPushed(app, event)
    scenario = app.AppScenarioDropDown.Value;

    % Shared FEM solver (same numerical method as main solver)
    % Uses T3 triangular elements, conduction + convection
    % Each scenario sets its own physics parameters and geometry,
    % then runs through the same FEM assembly and solve pipeline.

    switch scenario

        % SCENARIO 1: Electronic Device Cooling
        % A CPU chip (heat source at base) cooled by forced air
        % convection on the exposed surfaces.
        % Geometry: 20mm x 10mm silicon substrate (2D cross-section)
        case 'Electronic Device Cooling'

            % Physics Parameters
            k_si      = 148;      % Thermal conductivity of silicon
            h_forced   = 250;     % Forced-air convection coefficient
            T_ambient  = 25;      % Ambient air temperature [°C]
            T_chip_base = 85;     % Heat source (chip junction) temperature
            thick_dev  = 0.005;   % Device thickness (5 mm) [m]

            % Mesh Parameters
            Lx = 0.020; % Domain width [m] (20 mm)
            Ly = 0.010; % Domain height [m] (10 mm)
            nx = 10;    % Elements in x

```

```

ny = 5;          % Elements in y

titleStr = 'Electronic Device Cooling – Temperature (°C)';
xlabel    = 'X (m) [Chip Width]';
ylabel    = 'Y (m) [Chip Thickness]';
figNum    = 2;

% Boundary conditions:
% Bottom edge (y=0) → fixed T = T_chip_base (heat source)
% Top / Left / Right → convection with h_forced, T_ambient
fixedEdgeFn = @(x,y) (abs(y) < 1e-10);          % y = 0
convectEdgeFn = @(x,y) (abs(y-Ly)<1e-10) | ...    % y = Ly
                  (abs(x)<1e-10) | ...             % x = 0
                  (abs(x-Lx)<1e-10);              % x = Lx

T_fixed = T_chip_base;
T_inf    = T_ambient;

resultMsg = sprintf( ...
    ['Electronic Device Cooling\n' ...
    '  Material : Silicon\n' ...
    '  k = %.0f W/(m·K)\n' ...
    '  h = %.0f W/(m²·K)\n' ...
    '  T_base = %.0f °C | T_ambient = %.0f °C\n' ...
    '  Domain: %.0f mm × %.0f mm\n' ...
    '  → See Figure %d for temperature map.'], ...
    k_si, h_forced, T_chip_base, T_ambient, ...
    Lx*1e3, Ly*1e3, figNum);

% SCENARIO 2: Building Wall Heating
% A concrete wall with interior heating surface and
% exterior cold-air exposure (winter insulation study).
% Geometry: 300mm thick × 1000mm tall wall cross-section
case 'Building Wall Heating'

    % Physics Parameters
    k_concrete = 1.7;          % Thermal conductivity of concrete
    h_ext       = 20;          % Exterior wind-driven convection
    T_interior  = 20;          % Interior surface (heated side) [°C]
    T_exterior  = -10;         % Exterior ambient temperature [°C]
    thick_dev   = 1.0;         % Out-of-plane depth (1 m strip) [m]

    % Mesh Parameters
    Lx = 0.300; % Wall thickness [m] (300 mm)
    Ly = 1.000; % Wall height [m] (1000 mm)
    nx = 6;     % Elements in x (through thickness)
    ny = 10;    % Elements in y (along height)

    titleStr = 'Building Wall Heating – Temperature (°C)';
    xlabel    = 'X (m) [Through Wall Thickness]';
    ylabel    = 'Y (m) [Wall Height]';
    figNum    = 3;

    % Boundary conditions:

```

[W/(m·K)]

[W/(m²·K)]

```

% Left edge (x=0) → fixed T = T_interior (heated room side)
% Right edge (x=Lx) → convection h_ext, T_exterior (outside)
% Top / Bottom → adiabatic (no flux, default)
fixedEdgeFn = @(x,y) (abs(x) < 1e-10); % x = 0
(interior)
convectEdgeFn = @(x,y) (abs(x-Lx) < 1e-10); % x = Lx
(exterior)

T_fixed = T_interior;
T_inf = T_exterior;
h_forced = h_ext;
k_si = k_concrete;

resultMsg = sprintf( ...
    ['Building Wall Heating\n' ...
    ' Material : Concrete\n' ...
    ' k = %.1f W/(m·K)\n' ...
    ' h_ext = %.0f W/(m²·K)\n' ...
    ' T_interior = %.0f °C | T_exterior = %.0f °C\n' ...
    ' Wall: %.0f mm thick × %.0f mm tall\n' ...
    ' → See Figure %d for temperature map.'], ...
    k_concrete, h_ext, T_interior, T_exterior, ...
    Lx*1e3, Ly*1e3, figNum);

otherwise
    app.RealWorldResultsLabel.Text = 'Please select a valid
scenario.';

return
end

% SHARED FEM SOLVER (same numerical method as main code)
% T3 triangular elements – conduction + convection
% Build rectangular mesh of T3 elements (2 triangles / quad)
dx = Lx / nx;
dy = Ly / ny;

% Node coordinates
numNodes = (nx+1)*(ny+1);
nodeCoords = zeros(numNodes, 2);
idx = 1;
for ix = 0:nx
    for iy = 0:ny
        nodeCoords(idx,:) = [ix*dx, iy*dy];
        idx = idx + 1;
    end
end

nodeID = @(ix,iy) ix*(ny+1) + iy + 1; % 1-based

% Element connectivity (split each quad into 2 T3 triangles)
numElem = 2*nx*ny;
elemConn = zeros(numElem, 3);
eIdx = 1;
for ix = 0:nx-1
    for iy = 0:ny-1
        n1 = nodeID(ix, iy);

```

```

        n2 = nodeID(ix+1, iy);
        n3 = nodeID(ix+1, iy+1);
        n4 = nodeID(ix, iy+1);
        elemConn(eIdx,:) = [n1 n2 n3]; % lower-right triangle
        elemConn(eIdx+1,:) = [n1 n3 n4]; % upper-left triangle
        eIdx = eIdx + 2;
    end
end

% Material matrix
D = [k_si 0; 0 k_si];

% Identify boundary conditions
nDOF = numNodes;
isFixed = false(numNodes,1);
T_bc = zeros(numNodes,1);
for nd = 1:numNodes
    xn = nodeCoords(nd,1);
    yn = nodeCoords(nd,2);
    if fixedEdgeFn(xn, yn)
        isFixed(nd) = true;
        T_bc(nd) = T_fixed;
    end
end

% Global stiffness and force
KK = zeros(nDOF, nDOF);
FF = zeros(nDOF, 1);

for e = 1:numElem
    ni = elemConn(e,1);
    nj = elemConn(e,2);
    nk = elemConn(e,3);

    xi = nodeCoords(ni,:);
    xj = nodeCoords(nj,:);
    xk = nodeCoords(nk,:);

    % Element area
    Ae = 0.5 * abs(det([1 xi; 1 xj; 1 xk]));
    if Ae < 1e-20, continue; end

    % Shape function gradients
    b1 = (xj(2)-xk(2))/(2*Ae);
    b2 = (xk(2)-xi(2))/(2*Ae);
    b3 = (xi(2)-xj(2))/(2*Ae);
    c1 = (xk(1)-xj(1))/(2*Ae);
    c2 = (xi(1)-xk(1))/(2*Ae);
    c3 = (xj(1)-xi(1))/(2*Ae);

    B = [b1 b2 b3;
         c1 c2 c3];

    % Conduction matrix
    Kc = thick_dev * Ae * (B' * D * B);

```

```

% Convection on edges – check each edge of this element
Kh = zeros(3,3);
fh = zeros(3,1);
edges = [1 2; 2 3; 3 1]; % local node pairs per edge
locNodes = [ni nj nk];

for ed = 1:3
    ea = locNodes(edges(ed,1));
    eb = locNodes(edges(ed,2));
    xa = nodeCoords(ea,:);
    xb = nodeCoords(eb,:);

    % Midpoint test for convection edge
    xmid = 0.5*(xa + xb);
    if convectEdgeFn(xmid(1), xmid(2))
        Le = norm(xb - xa);
        % Convection stiffness (consistent)
        Kh_e = (h_forced * Le * thick_dev / 6) * [2 1; 1 2];
        fh_e = (h_forced * T_inf * Le * thick_dev / 2) * [1; 1];
        locs = edges(ed,:);
        Kh(locs, locs) = Kh(locs, locs) + Kh_e;
        fh(locs) = fh(locs) + fh_e;
    end
end

% Assemble into global system
dofs = [ni nj nk];
KK(dofs, dofs) = KK(dofs, dofs) + Kc + Kh;
FF(dofs) = FF(dofs) + fh;
end

% Apply fixed-temperature BCs (penalty / direct substitution)
fixedDOFs = find(isFixed);
for nd = fixedDOFs'
    FF = FF - KK(:, nd) * T_bc(nd);
end
for nd = fixedDOFs'
    KK(nd, :) = 0;
    KK(:, nd) = 0;
    KK(nd, nd) = 1;
    FF(nd) = T_bc(nd);
end

% Solve
T_sol = KK \ FF;

% --- Plot ---
figure(figNum)
patch('Faces', elemConn, 'Vertices', nodeCoords, ...
    'FaceVertexCData', T_sol, ...
    'FaceColor', 'interp', 'EdgeColor', 'none')
colorbar
colormap(jet)
title(titleStr)

```



```

xlabel(xLabel)
ylabel(yLabel)
set(gcf, 'Position', [450 100 560 420])

% Add max/min annotation
T_max = max(T_sol);
T_min = min(T_sol);
annotation_str = sprintf('T_{max}=%.1f°C\nT_{min}=%.1f°C', T_max, T_min);
text(0.02, 0.95, annotation_str, 'Units','normalized', ...
    'FontSize', 10, 'Color', 'w', 'FontWeight','bold', ...
    'VerticalAlignment','top')

% Update result label in UI
app.RealWorldResultsLabel.Text = resultMsg;
end

end

% Component initialization
methods (Access = private)

% Create UIFigure and components
function createComponents(app)

    % Get the file path for locating images
    pathToMLAPP = fileparts(mfilename('Project'));

    % Create UIFigure and hide until all components are created
    app.UIFigure = uifigure('Visible', 'off');
    app.UIFigure.Position = [100 100 640 650];
    app.UIFigure.Name = 'MATLAB App';

    % Create Image
    app.Image = uiimage(app.UIFigure);
    app.Image.BackgroundColor = [0.1882 0.0118 0.1882];
    app.Image.Position = [1 1 640 650];
    app.Image.ImageSource = fullfile(pathToMLAPP,
'360_F_487461421_L0q4P5iS2nTd4JfRlYJjKlgLRPZXP4sY.png');

    % Create ThermalConductivityLabel
    app.ThermalConductivityLabel = uilabel(app.UIFigure);
    app.ThermalConductivityLabel.HorizontalAlignment = 'center';
    app.ThermalConductivityLabel.FontSize = 14;
    app.ThermalConductivityLabel.FontWeight = 'bold';
    app.ThermalConductivityLabel.FontColor = [1 0.6 0];
    app.ThermalConductivityLabel.Position = [45 428 116 51];
    app.ThermalConductivityLabel.Text = {'Thermal '; 'Conductivity:'};

    % Create ThermalConductivityEditField
    app.ThermalConductivityEditField = uieditfield(app.UIFigure, 'numeric');
    app.ThermalConductivityEditField.BackgroundColor = [1 0.6 0];
    app.ThermalConductivityEditField.FontColor = [1 1 1];
    app.ThermalConductivityEditField.Position = [29 378 149 51];

    % Create HeatConvectionCoefficientLabel

```

```

app.HeatConvectionCoefficientLabel = uilabel(app.UIFigure);
app.HeatConvectionCoefficientLabel.HorizontalAlignment = 'center';
app.HeatConvectionCoefficientLabel.FontSize = 14;
app.HeatConvectionCoefficientLabel.FontWeight = 'bold';
app.HeatConvectionCoefficientLabel.FontColor = [1 0.6 0];
app.HeatConvectionCoefficientLabel.Position = [464 435 162 34];
app.HeatConvectionCoefficientLabel.Text = {'Heat '; 'Convection
Coefficient: '};

% Create HeatConvectionCoefficientEditField
app.HeatConvectionCoefficientEditField = uieditfield(app.UIFigure,
'numeric');
app.HeatConvectionCoefficientEditField.BackgroundColor = [1 0.6 0];
app.HeatConvectionCoefficientEditField.FontColor = [1 1 1];
app.HeatConvectionCoefficientEditField.FontSize = 14;
app.HeatConvectionCoefficientEditField.FontWeight = 'bold';
app.HeatConvectionCoefficientEditField.Position = [464 378 162 50];

% Create NumberofBlocksEnterbetween1to4Label
app.NumberofBlocksEnterbetween1to4Label = uilabel(app.UIFigure);
app.NumberofBlocksEnterbetween1to4Label.HorizontalAlignment = 'right';
app.NumberofBlocksEnterbetween1to4Label.FontSize = 14;
app.NumberofBlocksEnterbetween1to4Label.FontColor = [1 1 1];
app.NumberofBlocksEnterbetween1to4Label.Position = [160 242 115 52];
app.NumberofBlocksEnterbetween1to4Label.Text = {'Number of Blocks';
'(Enter between '; '1 to 4): '};

% Create NumberofBlocksEnterbetween1to4EditField
app.NumberofBlocksEnterbetween1to4EditField = uieditfield(app.UIFigure,
'numeric');
app.NumberofBlocksEnterbetween1to4EditField.BackgroundColor = [0.1 0.05
0.15];
app.NumberofBlocksEnterbetween1to4EditField.FontColor = [1 1 1];
app.NumberofBlocksEnterbetween1to4EditField.FontWeight = 'bold';
app.NumberofBlocksEnterbetween1to4EditField.Position = [290 242 104 48];

% Create ThicknessinmmEditFieldLabel
app.ThicknessinmmEditFieldLabel = uilabel(app.UIFigure);
app.ThicknessinmmEditFieldLabel.HorizontalAlignment = 'right';
app.ThicknessinmmEditFieldLabel.FontSize = 14;
app.ThicknessinmmEditFieldLabel.FontColor = [1 0.5 0.3];
app.ThicknessinmmEditFieldLabel.Position = [26 221 119 22];
app.ThicknessinmmEditFieldLabel.Text = 'Thickness(in mm):';

% Create ThicknessinmmEditField
app.ThicknessinmmEditField = uieditfield(app.UIFigure, 'numeric');
app.ThicknessinmmEditField.BackgroundColor = [1 0.5 0.3];
app.ThicknessinmmEditField.FontColor = [1 1 1];
app.ThicknessinmmEditField.Position = [13 180 148 42];

% Create BoundedTemperatureInitialLabel
app.BoundedTemperatureInitialLabel = uilabel(app.UIFigure);
app.BoundedTemperatureInitialLabel.HorizontalAlignment = 'center';
app.BoundedTemperatureInitialLabel.FontSize = 14;
app.BoundedTemperatureInitialLabel.FontColor = [1 0.5 0.3];

```

```

app.BoundedTemperatureInitialLabel.Position = [473 221 143 34];
app.BoundedTemperatureInitialLabel.Text = {'Bounded Temperature';
'(Initial):'};

% Create BoundedTemperatureInitialEditField
app.BoundedTemperatureInitialEditField = uieditfield(app.UIFigure,
'numeric');
app.BoundedTemperatureInitialEditField.BackgroundColor = [1 0.5 0.3];
app.BoundedTemperatureInitialEditField.FontColor = [1 1 1];
app.BoundedTemperatureInitialEditField.Position = [473 180 143 42];

% Create ShowPlotButton
app.ShowPlotButton = uibutton(app.UIFigure, 'push');
app.ShowPlotButton.ButtonPushedFcn = createCallbackFcn(app,
@ShowPlotButtonPushed, true);
app.ShowPlotButton.FontName = 'Bell MT';
app.ShowPlotButton.FontSize = 24;
app.ShowPlotButton.FontWeight = 'bold';
app.ShowPlotButton.BackgroundColor = [0.9 0.4 0];
app.ShowPlotButton.FontColor = [1 1 1];
app.ShowPlotButton.Position = [233 321 176 58];
app.ShowPlotButton.Text = 'Show Plot: ';

% NEW: Real-World Application Panel
% Positioned below the existing controls

% Panel container
app.RealWorldPanel = uipanel(app.UIFigure);
app.RealWorldPanel.Title = 'Real-World Heat Transfer Applications';
app.RealWorldPanel.FontSize = 12;
app.RealWorldPanel.FontWeight = 'bold';
app.RealWorldPanel.ForegroundColor = [1 1 1];
app.RealWorldPanel.BackgroundColor = [0.12 0.06 0.18];
app.RealWorldPanel.Position = [10 1 620 115];

% Scenario label
app.AppScenarioLabel = uilabel(app.RealWorldPanel);
app.AppScenarioLabel.Text = 'Select Scenario: ';
app.AppScenarioLabel.FontSize = 11;
app.AppScenarioLabel.FontWeight = 'bold';
app.AppScenarioLabel.FontColor = [0 1 1];
app.AppScenarioLabel.Position = [5 60 110 22];

% Scenario drop-down
app.AppScenarioDropDown = uidropdown(app.RealWorldPanel);
app.AppScenarioDropDown.Items = {'Electronic Device Cooling', 'Building
Wall Heating'};
app.AppScenarioDropDown.Value = 'Electronic Device Cooling';
app.AppScenarioDropDown.FontSize = 11;
app.AppScenarioDropDown.Position = [118 57 210 26];

% Run button
app.RunRealWorldButton = uibutton(app.RealWorldPanel, 'push');
app.RunRealWorldButton.Text = 'Run Application';

```

```

        app.RunRealWorldButton.ButtonPushedFcn = createCallbackFcn(app,
@RunRealWorldButtonPushed, true);
        app.RunRealWorldButton.FontSize = 11;
        app.RunRealWorldButton.FontWeight = 'bold';
        app.RunRealWorldButton.FontColor = [0.6353 0.0784 0.1843];
        app.RunRealWorldButton.Position = [340 55 120 30];

        % Results label (shows scenario summary after run)
        app.RealWorldResultsLabel = uilabel(app.RealWorldPanel);
        app.RealWorldResultsLabel.Text = 'Results will appear here after running
a scenario.';
        app.RealWorldResultsLabel.FontSize = 8;
        app.RealWorldResultsLabel.FontColor = [1 1 0.5];
        app.RealWorldResultsLabel.Position = [485 2 135 103];
        app.RealWorldResultsLabel.WordWrap = 'on';

        % Show the figure after all components are created
        app.UIFigure.Visible = 'on';
    end
end

% App creation and deletion
methods (Access = public)

    % Construct app
    function app = EEE_PROJECT_FROM_95_AND_96

        % Create UIFigure and components
        createComponents(app)

        % Register the app with App Designer
        registerApp(app, app.UIFigure)

        if nargin == 0
            clear app
        end
    end

    % Code that executes before app deletion
    function delete(app)

        % Delete UIFigure when app is deleted
        delete(app.UIFigure)
    end
end
end

```

Problems we have faced:

During the development of this project, we encountered several challenges that affected both implementation and accuracy. One significant issue was handling the

coordinate mapping of triangular (T3) elements, as small mistakes in node ordering led to incorrect area calculations and, consequently, wrong element matrices. Ensuring continuity between adjacent elements was also difficult, sometimes resulting in non-physical temperature variations across the domain. Selecting appropriate boundary nodes manually posed another challenge, as incorrect identification caused errors in applying boundary conditions. Balancing accuracy and computation time was critical—finer meshes improved results but significantly increased simulation time and memory usage. Numerical errors such as round-off errors and matrix conditioning issues occasionally affected solution stability. In MATLAB, performance was impacted by the handling of sparse versus full matrices, requiring careful optimization of loops and avoidance of redundant calculations. Verifying results was also challenging due to the absence of simple analytical solutions for complex cases, and plotting smooth temperature contours with correct interpolation between nodes required additional effort.

In addition to these technical difficulties, applying the model to real-world scenarios revealed further complications. Material properties in practical applications, such as silicon chips or concrete walls, are often non-uniform and temperature-dependent, which can lead to deviations from the idealized simulation. Irregular geometries and complex boundary conditions in real devices make meshing and node assignment more intricate. Environmental factors, such as fluctuating ambient temperatures or airflow variations, introduce uncertainties that must be accounted for in modeling. Finally, scaling the simulation for industrial-scale problems can be computationally intensive, requiring efficient algorithms and high-performance computing resources. Through repeated testing, debugging, and optimization, we gradually addressed these issues, improving both the accuracy and robustness of our model and preparing it for practical engineering applications.

Future work:

- Extension of the present model to transient (time-dependent) heat conduction analysis.
- Implementation of higher-order elements (e.g., quadratic elements) to enhance solution accuracy.
- Incorporation of adaptive mesh refinement techniques for improved efficiency and precision.
- Consideration of nonlinear and temperature-dependent material properties.
- Application of more advanced and realistic boundary conditions.
- Optimization of the MATLAB implementation using sparse matrix techniques.
- Utilization of parallel computing methods to improve computational performance.
- Expansion of the model to address three-dimensional heat conduction problems.
- Enhancement of post-processing and visualization methods for clearer result interpretation.
- Validation of numerical results through comparison with experimental or benchmark data.

References:

- Standard textbooks on Heat Transfer and Finite Element Method (FEM)
- MATLAB Official Documentation
- Relevant research papers and journal articles on FEM and heat conduction
- Course lecture notes and instructional materials
- Online educational resources for FEM and MATLAB implementation

- Course lecture notes & lab sheets, BUET

Conclusion:

This project successfully developed a comprehensive Finite Element Method (FEM) framework for solving two-dimensional steady-state heat conduction problems, implemented fully in MATLAB. The application includes detailed pre-processing steps such as mesh generation, element connectivity, and assembly of global stiffness and force matrices, as well as rigorous enforcement of Dirichlet and Neumann boundary conditions. The model was validated against analytical solutions and benchmark scenarios, demonstrating high accuracy and computational efficiency. Key technical challenges, including the application of boundary temperatures, convection effects, and multi-block geometries, were systematically addressed, resulting in a robust and adaptable solver. Furthermore, the extension to real-world scenarios, such as electronic device cooling and building wall heating, illustrates the flexibility of the FEM framework in handling practical engineering problems with varying material properties, convection coefficients, and complex geometries. Overall, the project not only achieved reliable numerical predictions but also establishes a strong foundation for future enhancements, including transient analysis, nonlinear materials, and three-dimensional heat transfer simulations, thereby providing a valuable tool for advanced thermal analysis in engineering applications.